

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



ĐỀ CƯƠNG CHUYÊN ĐỀ CƠ SỞ
**NGHIÊN CỨU ỨNG DỤNG HỌC SÂU TRONG
BÀI TOÁN PHÁT HIỆN XÂM NHẬP MẠNG TRÁI PHÉP**

Sinh viên thực hiện:

Nguyễn Hồng Phong – MSSV: AT190540

Người hướng dẫn:

TS. Nguyễn Mạnh Thắng

Khoa An toàn thông tin – Học viện Kỹ thuật mật mã

Hà Nội, 2025

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



ĐỀ CƯƠNG CHUYÊN ĐỀ CƠ SỞ
**NGHIÊN CỨU ỨNG DỤNG HỌC SÂU TRONG
BÀI TOÁN PHÁT HIỆN XÂM NHẬP MẠNG TRÁI PHÉP**

Sinh viên thực hiện:

Nguyễn Hồng Phong – MSSV: AT190540

Người hướng dẫn:

TS. Nguyễn Mạnh Thắng

Khoa An toàn thông tin – Học viện Kỹ thuật mật mã

Hà Nội, 2025

MỤC LỤC

Danh mục kí hiệu và viết tắt.....	iii
Danh mục hình vẽ.....	iv
Danh mục bảng.....	v
Lời cảm ơn	vi
Lời nói đầu	vii
Chương 1. Tổng quan về hệ thống phát hiện xâm nhập	1
1.1. Giới thiệu về hệ thống phát hiện và ngăn chặn xâm nhập (IDPS).....	1
1.1.1. Khái niệm và vai trò chiến lược trong an ninh mạng.....	1
1.1.2. Phân loại IDPS	1
1.2. Các phương pháp phát hiện xâm nhập.....	3
1.2.1. Phương pháp dựa trên chữ ký	3
1.2.2. Phương pháp dựa trên bất thường	3
1.3. Thách thức và nhu cầu áp dụng học máy.....	4
1.4. Kết luận chương 1	5
Chương 2. Mô hình học sâu và bộ dữ liệu CIC-IDS2017.....	6
2.1. Các mô hình học sâu nền tảng	6
2.1.1. Mạng Perceptron đa lớp.....	6
2.1.2. Mạng nơ-ron sâu - Deep Neural Network.....	7
2.1.3. Mạng nơ-ron tích chập một chiều	8
2.1.4. Mối quan hệ giữa MLP, DNN và CNN.....	9
2.2. Các thành phần kỹ thuật cốt lõi và phương pháp đánh giá.....	11
2.2.1. Hàm kích hoạt.....	11
2.2.2. Hàm mất mát.....	12
2.2.3. Thuật toán tối ưu hóa và kỹ thuật chống quá khớp.....	13
2.2.4. Các độ đo đánh giá hiệu năng.....	13
2.3. Bộ dữ liệu CIC-IDS-2017	15
2.3.1. Giới thiệu tổng quan và lý do lựa chọn	15
2.3.2. Mô tả các loại tấn công trong bộ dữ liệu	16
2.4. Phân tích khám phá bộ dữ liệu.....	17
2.4.1. Phân tích đặc điểm và cấu trúc bộ dữ liệu.....	17
2.4.2. Đánh giá chất lượng và mối quan hệ giữa các đặc trưng.....	19
2.4.3. Phân tích hành vi và phân bố dữ liệu.....	22
2.5. Kết luận chương 2	25
Chương 3. Triển khai thực nghiệm	26
3.1. Môi trường và quy trình thực nghiệm.....	26
3.1.1. Môi trường thực nghiệm.....	26
3.1.2. Quy trình tổng thể.....	26
3.2. Tiền xử lý dữ liệu chung	26
3.2.1. Tích hợp và làm sạch dữ liệu sơ bộ.....	26
3.2.2. Chiến lược tối ưu hóa đặc trưng	28
3.2.3. Phân chia tập và chuẩn hóa tập dữ liệu.....	29
3.3. Kịch bản 1: Phát hiện xâm nhập - Phân loại nhị phân.....	31

3.3.1.	<i>Xử lý dữ liệu cho bài toán nhị phân</i>	31
3.3.2.	<i>Triển khai mô hình Deep Neural Network</i>	32
3.3.3.	<i>Triển khai mô hình 1D-CNN</i>	37
3.3.4.	<i>Đánh giá so sánh và tiểu kết cho kịch bản nhị phân</i>	42
3.4.	<i>Kịch bản 2: Phân loại dạng tấn công - Phân loại đa lớp</i>	43
3.4.1.	<i>Xử lý dữ liệu cho bài toán phân loại đa lớp</i>	43
3.4.2.	<i>Triển khai mô hình DNN cho phân loại đa lớp</i>	45
3.4.3.	<i>Triển khai mô hình 1D-CNN đa lớp</i>	47
3.4.4.	<i>Đánh giá so sánh và tiểu kết cho kịch bản phân loại đa lớp</i>	49
	Kết luận	51
	Tài liệu tham khảo	53

DANH MỤC KÍ HIỆU VÀ VIẾT TẮT

1D-CNN	1D Convolutional Neural Network
AUC	Area Under the Curve
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
DNN	Deep Neural Network
HIDS	Host-based IDPS
IDPS	Intrusion Detection and Prevention System
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
MLP	Multi-Layer Perceptron
NIDS	Network-based IDPS
PCAP	Packet Capture
ReLU	Rectified Linear Unit
ROC	Receiver Operating Characteristic
SMOTE	Synthetic Minority Over-sampling Technique

DANH MỤC HÌNH VẼ

Hình 2-1 Quy trình xây dựng và xử lý bộ dữ liệu CIC-IDS-2017.....	15
Hình 2-2. Ma trận tương quan giữa các đặc trưng của bộ dữ liệu	20
Hình 2-3.10 đặc trưng có ảnh hưởng lớn nhất đến nhãn.....	21
Hình 2-4. Phân bố Độ dài trung bình gói tin theo từng loại tấn công.....	22
Hình 2-5. Phân bố Thời gian trung bình giữa các gói tin (Flow IAT Mean).....	23
Hình 2-6. Phân bố Flow Bytes/s theo từng loại tấn công (Thang đo Log)	23
Hình 2-7. Phân bố số lượng các lớp tấn công trong bộ dữ liệu	24
Hình 3-1 Biểu đồ hộp phân phối của cột Flow Bytes/s (a) và Flow Packets/s (b) .	27
Hình 3-2. Biểu đồ hội tụ của hàm mất mát và độ chính xác huấn luyện (DNN)....	35
Hình 3-3. Ma trận nhầm lẫn của mô hình DNN.....	36
Hình 3-4. Đường cong ROC và diện tích dưới đường cong AUC	37
Hình 3-5. Biểu đồ hội tụ của hàm mất mát và độ chính xác huấn luyện (1D-CNN)	40
Hình 3-6. Ma trận nhầm lẫn của mô hình 1D-CNN.....	41
Hình 3-7. Đường cong ROC và chỉ số AUC.	41
Hình 3-8. Biểu đồ quá trình hội tụ (DNN – đa lớp).....	45
Hình 3-9. Ma trận nhầm lẫn của mô hình DNN (phân loại đa lớp).....	47
Hình 3-10. Biểu đồ quá trình hội tụ của 1D-CNN (đa lớp)	48
Hình 3-11. Ma trận nhầm lẫn mô hình 1D-CNN (đa lớp)	49

DANH MỤC BẢNG

Bảng 2-1. Bảng so sánh mô hình mạng nơ-ron sâu với mạng tính chập	10
Bảng 2-2. Danh sách các tệp dữ liệu thành phần của CIC-IDS-2017.....	17
Bảng 3-1. Báo cáo chỉ số phân loại trên tập kiểm thử	35
Bảng 3-2. Bảng trình bày các chỉ số định lượng chi tiết (1D-CNN)	40
Bảng 3-3. So sánh kết quả 2 mô hình trong phân loại nhị phân	42
Bảng 3-4. Báo cáo phân loại chi tiết cho 5 nhóm nhãn (DNN)	46
Bảng 3-5. Báo cáo phân loại chi tiết cho 5 nhóm nhãn (1D-CNN).....	48
Bảng 3-6. Bảng so sánh hiệu năng giữa DNN và 1D-CNN (Đa lớp).....	49

LỜI CẢM ƠN

Trong quá trình thực hiện đề tài này, em đã nhận được sự giúp đỡ tận tình của cán bộ hướng dẫn là TS. Nguyễn Mạnh Thắng – Giảng viên Khoa An toàn thông tin Học viện Kỹ thuật Mật mã, sự quan tâm sâu sát của cán bộ Hệ quản lý học viên quốc tế, sự động viên của người thân và bạn bè.

Xin cảm ơn tất cả mọi người đã tạo những điều kiện tốt nhất để em có thể hoàn thành đề tài này!

SINH VIÊN THỰC HIỆN ĐỀ TÀI

Nguyễn Hồng Phong

LỜI NÓI ĐẦU

Trong bối cảnh an ninh mạng hiện đại, các giải pháp phát hiện xâm nhập (IDS) truyền thống dựa trên chữ ký đang bộc lộ hạn chế lớn: thụ động và bất lực trước các cuộc tấn công chưa biết (Zero-day) hoặc mã độc đa hình. Khoảng trống thời gian cập nhật chữ ký thủ công chính là điểm yếu chí mạng mà tin tặc khai thác. Do đó, việc ứng dụng Trí tuệ nhân tạo, cụ thể là Học sâu (Deep Learning), với khả năng tự động trích xuất đặc trưng phức tạp từ dữ liệu mạng, là nhu cầu cấp thiết để nâng cao khả năng phòng thủ chủ động.

Xuất phát từ thực tiễn trên, đề án "Nghiên cứu ứng dụng Học sâu trong bài toán phát hiện xâm nhập mạng trái phép" tập trung giải quyết các mục tiêu cốt lõi sau:

- Nghiên cứu lý thuyết: Hệ thống hóa kiến trúc các mô hình học sâu nền tảng gồm DNN và mạng tích chập 1D-CNN.
- Xử lý dữ liệu: Phân tích bộ dữ liệu chuẩn mực CIC-IDS-2017, tập trung giải quyết bài toán mất cân bằng mẫu nghiêm trọng bằng kỹ thuật SMOTE và Undersampling.
- Xây dựng mô hình: Đề xuất và triển khai mô hình 1D-CNN để khai thác đặc trưng không gian của gói tin, thực nghiệm trên hai bài toán: phát hiện xâm nhập (nhị phân) và định danh loại tấn công (đa lớp), nhằm chứng minh hiệu năng vượt trội so với các phương pháp truyền thống.

Kết quả của đề án hy vọng sẽ đóng góp một cơ sở thực nghiệm tin cậy cho việc ứng dụng Deep Learning vào thực tế giám sát an ninh mạng, khắc phục được tỷ lệ báo động giả và nâng cao khả năng phát hiện các cuộc tấn công phức tạp.

SINH VIÊN THỰC HIỆN ĐỀ TÀI

Nguyễn Hồng Phong

CHƯƠNG 1. TỔNG QUAN VỀ HỆ THỐNG PHÁT HIỆN XÂM NHẬP

1.1. Giới thiệu về hệ thống phát hiện và ngăn chặn xâm nhập (IDPS)

1.1.1. Khái niệm và vai trò chiến lược trong an ninh mạng

Trong bối cảnh an ninh mạng hiện đại, các mối đe dọa không còn giới hạn ở mã độc đơn giản mà đã phát triển thành những cuộc tấn công phức tạp nhắm vào tính bảo mật, toàn vẹn và sẵn sàng của hệ thống tài nguyên. Để đối phó với thách thức này, Hệ thống Phát hiện và Ngăn chặn Xâm nhập (IDPS) ra đời như một giải pháp bảo mật tích hợp, bao trùm hai thành phần công nghệ có mối quan hệ mật thiết nhưng khác biệt về phương thức hoạt động là IDS và IPS.

Về cơ chế vận hành, Hệ thống Phát hiện Xâm nhập (IDS) hoạt động ở chế độ thụ động, thường được kết nối vào cổng sao chép dữ liệu (SPAN port hoặc Mirror port) để giám sát bản sao của lưu lượng mạng mà không làm ảnh hưởng đến hiệu năng đường truyền chính. Nhiệm vụ của IDS là phân tích gói tin để tìm kiếm các dấu hiệu bất thường hoặc các mẫu tấn công đã biết và gửi cảnh báo đến quản trị viên, tuy nhiên nó không có khả năng tự động chặn luồng dữ liệu độc hại. Ngược lại, Hệ thống Ngăn chặn Xâm nhập (IPS) được triển khai chắn ngang luồng dữ liệu (Inline mode), buộc mọi gói tin đi vào hoặc ra khỏi hệ thống mạng đều phải đi xuyên qua bộ lọc kiểm soát. Vị trí chiến lược này cho phép IPS vừa phân tích gói tin, vừa ra quyết định can thiệp thời gian thực như hủy bỏ gói tin, ngắt kết nối TCP hoặc chặn địa chỉ IP nguồn ngay khi phát hiện vi phạm.

Sự hiện diện của IDPS đóng vai trò thiết yếu trong chiến lược phòng thủ chiều sâu (Defense-in-Depth), lấp đầy những khoảng trống an ninh mà các thiết bị tường lửa truyền thống để lại. Khác với tường lửa chỉ lọc dựa trên thông tin địa chỉ và cổng, IDPS sử dụng công nghệ Phân tích sâu gói tin (DPI) để kiểm tra nội dung tại tầng ứng dụng, nhờ đó phát hiện được các tấn công ẩn trong giao thức hợp lệ như SQL Injection hay XSS. Đồng thời, khả năng phản ứng tự động của hệ thống giúp ngăn chặn tức thời các cuộc tấn công tốc độ cao như Botnet, đồng thời cung cấp nguồn dữ liệu nhật ký chi tiết phục vụ công tác điều tra số và phân tích nguyên nhân sự cố sau này.

1.1.2. Phân loại IDPS

Dựa trên phạm vi giám sát, vị trí triển khai và nguồn dữ liệu thu thập được, các hệ thống phát hiện xâm nhập thường được phân loại thành hai nhóm chính: Hệ thống phát hiện xâm nhập dựa trên mạng (Network-based IDPS - NIDS) và Hệ thống

phát hiện xâm nhập dựa trên máy chủ (Host-based IDPS - HIDS). Mỗi loại hình đều có những đặc điểm kỹ thuật, ưu điểm và hạn chế riêng, phù hợp với các chiến lược bảo vệ khác nhau.

a. Hệ thống phát hiện xâm nhập dựa trên mạng (NIDS)

Mô hình NIDS được thiết kế để giám sát lưu lượng trên toàn bộ phân đoạn mạng hoặc hệ thống mạng của tổ chức thông qua các cảm biến (sensors) được triển khai tại những điểm nút chiến lược như sau bộ định tuyến biên, vùng DMZ hoặc tại các switch trung tâm. Về cơ chế hoạt động, NIDS kết nối vào hạ tầng mạng thông qua cổng phản chiếu (Mirror Port) hoặc thiết bị chia tín hiệu, cho phép các giao diện mạng hoạt động ở chế độ hỗn tạp để thu thập toàn bộ gói tin đi qua đường truyền một cách thụ động. Hệ thống sẽ phân tích đồng thời cả phần tiêu đề và nội dung của gói tin để phát hiện các dấu hiệu tấn công trong luồng dữ liệu mà không làm gián đoạn hoạt động mạng.

Ưu điểm nổi bật của NIDS nằm ở khả năng giám sát diện rộng và tiết kiệm chi phí, khi chỉ cần một vài thiết bị có thể bảo vệ cả một mạng lưới lớn, đồng thời khả năng hoạt động thụ động giúp nó khó bị kẻ tấn công phát hiện và đảm bảo tính toàn vẹn của bằng chứng số. Tuy nhiên, hạn chế lớn nhất của mô hình này là sự bất lực trước các luồng dữ liệu đã mã hóa như HTTPS hay VPN nếu thiếu giải pháp giải mã trung gian, cũng như nguy cơ bị quá tải (bottleneck) dẫn đến bỏ sót gói tin khi phải xử lý lưu lượng mạng tốc độ cao.

b. Hệ thống phát hiện xâm nhập dựa trên máy chủ (HIDS)

Khác với NIDS giám sát đường truyền, HIDS tập trung bảo vệ trực tiếp tại "điểm đến" bằng cách cài đặt các phần mềm đại lý (agent) trên từng máy chủ hoặc thiết bị đầu cuối. Hệ thống này đi sâu phân tích các hoạt động nội tại của hệ điều hành thông qua việc giám sát nhật ký hệ thống và ứng dụng, kiểm tra tính toàn vẹn tệp tin (FIM) để phát hiện sự thay đổi trái phép, và theo dõi các lời gọi hệ thống nhằm nhận diện hành vi leo thang đặc quyền.

Lợi thế chiến lược của HIDS là khả năng tiếp cận dữ liệu sau khi hệ điều hành đã giải mã, giúp phát hiện hiệu quả các cuộc tấn công qua kênh mã hóa mà NIDS thường bỏ qua, đồng thời nhận diện được các mối đe dọa nội bộ từ người dùng có quyền truy cập hợp lệ. Hơn nữa, HIDS có thể xác nhận chính xác xem một cuộc tấn công có thành công hay không dựa trên các thay đổi thực tế của hệ thống. Mặc dù vậy, việc triển khai HIDS đối mặt với thách thức lớn về mặt quản trị khi phải cài đặt trên quy mô lớn, đồng thời việc sử dụng chung tài nguyên phần cứng có thể làm

giảm hiệu suất của máy chủ ứng dụng và rủi ro bị vô hiệu hóa nếu kẻ tấn công chiếm được quyền kiểm soát cao nhất.

1.2. Các phương pháp phát hiện xâm nhập

Để một hệ thống IDPS hoạt động hiệu quả, "bộ não" trung tâm của nó (Detection Engine) cần được trang bị các thuật toán xử lý chuyên biệt nhằm phân biệt chính xác giữa lưu lượng mạng hợp lệ và các hoạt động tấn công. Trong lịch sử phát triển của lĩnh vực an toàn thông tin, các phương pháp này đã không ngừng tiến hóa để đối phó với sự gia tăng về độ phức tạp và tính ẩn danh của các cuộc tấn công mạng.

1.2.1. Phương pháp dựa trên chữ ký

Phương pháp phát hiện dựa trên chữ ký (Signature-based Detection), là kỹ thuật nền tảng và phổ biến nhất trong các hệ thống an ninh mạng thương mại hiện nay như Snort hay Suricata. Nguyên lý hoạt động dựa trên việc so sánh dữ liệu gói tin hoặc nhật ký hệ thống với cơ sở dữ liệu các mẫu tấn công đã biết (chữ ký). Nếu dữ liệu đầu vào trùng khớp với bất kỳ mẫu nào trong danh sách đen, hệ thống sẽ xác định đó là xâm nhập và lập tức kích hoạt cảnh báo hoặc ngăn chặn.

Để đảm bảo chính xác, hệ thống thực hiện giải mã, tái lập luồng dữ liệu và sử dụng các thuật toán tìm kiếm chuỗi hiệu năng cao như Aho-Corasick để quét chữ ký. Các chữ ký này có thể là dạng chuỗi đơn giản trong payload hoặc dạng hỗn hợp có trạng thái, giúp nhận diện chuỗi sự kiện phức tạp như rà quét cổng hay tấn công brute-force.

Ưu điểm của phương pháp là hiệu suất xử lý cao và tỷ lệ báo động giả thấp. Tuy nhiên, hạn chế lớn là sự bất lực trước các tấn công Zero-day và mã độc đa hình. Do phụ thuộc hoàn toàn vào cơ sở dữ liệu "đã biết", hệ thống sẽ bỏ qua mọi mối đe dọa mới khi chữ ký chưa kịp cập nhật, tạo ra lỗ hổng bảo vệ trước các chiến dịch tấn công mới.

1.2.2. Phương pháp dựa trên bất thường

Tiếp cận từ một triết lý hoàn toàn đối lập với phương pháp dựa trên chữ ký, phương pháp phát hiện bất thường (Anomaly-based Detection), hay còn gọi là phát hiện dựa trên hành vi, tập trung vào việc định nghĩa thế nào là trạng thái "bình thường" của hệ thống thay vì cố gắng liệt kê mọi dấu hiệu tấn công. Quy trình hoạt động gồm hai giai đoạn: Huấn luyện để xây dựng "Hồ sơ hành vi chuẩn" từ dữ liệu

sạch, và Phát hiện để so sánh lưu lượng thực tế với hồ sơ này. Mọi hành vi sai lệch vượt ngưỡng cho phép so với chuẩn đều bị đánh dấu là nguy cơ tiềm ẩn.

Về kỹ thuật, phương pháp này thường sử dụng thống kê hoặc phổ biến nhất là Học máy (Machine Learning) để tự động học các mẫu hành vi phức tạp. Ưu điểm chiến lược là khả năng phát hiện được các cuộc tấn công Zero-day và biến thể mã độc mới mà phương pháp chữ ký thường bỏ sót, do hành vi khai thác mạng của chúng vẫn khác biệt so với người dùng thông thường.

Tuy nhiên, phương pháp này gặp thách thức về tỷ lệ báo động giả cao do dễ nhầm lẫn các hành vi hợp lệ nhưng hiếm gặp với hành vi tấn công. Ngoài ra, chi phí tính toán lớn cho việc xử lý thời gian thực và nguy cơ bị "đầu độc" dữ liệu trong quá trình huấn luyện cũng là những rào cản kỹ thuật chính.

1.3. Thách thức và nhu cầu áp dụng học máy

Sự phát triển của công nghệ không chỉ mang lại lợi ích cho người dùng hợp pháp mà còn trang bị cho tội phạm mạng những công cụ tấn công ngày càng tinh vi, tự động hóa cao và đa hình. Trong bối cảnh đó, các hệ thống IDPS truyền thống dựa trên chữ ký đang dần bộc lộ bản chất thụ động và hạn chế về khả năng phản ứng. Vấn đề cốt lõi nằm ở việc cơ chế so khớp mẫu chỉ có thể phát hiện được những mối đe dọa "đã biết" nằm trong cơ sở dữ liệu. Đối với các lỗ hổng Zero-day chưa có bản vá hoặc các loại mã độc sử dụng kỹ thuật đa hình (Polymorphism) để liên tục thay đổi cấu trúc mã sau mỗi lần lây nhiễm, phương pháp truyền thống gần như trở nên vô hiệu. Hơn nữa, quy trình cập nhật chữ ký thủ công luôn tồn tại một "khoảng trống thời gian" nguy hiểm giữa thời điểm cuộc tấn công nổ ra và thời điểm hệ thống được cập nhật, khiến hạ tầng mạng hoàn toàn không được bảo vệ trước các chiến dịch tấn công có chủ đích.

Để giải quyết bài toán phát hiện các cuộc tấn công chưa biết và giảm thiểu tỷ lệ báo động giả của các phương pháp thống kê cũ, xu hướng nghiên cứu hiện đại đã chuyển dịch mạnh mẽ sang ứng dụng Trí tuệ nhân tạo. Trong khi các thuật toán Học máy truyền thống (như SVM, Decision Tree) vẫn gặp hạn chế do phụ thuộc nhiều vào quá trình trích xuất đặc trưng thủ công của chuyên gia, thì Học sâu (Deep Learning) nổi lên như một giải pháp ưu việt hơn hẳn. Lợi thế cạnh tranh lớn nhất của Học sâu là khả năng tự động trích xuất đặc trưng (Automatic Feature Extraction) từ dữ liệu thô thông qua các lớp ẩn nhiều tầng, cho phép mô hình tự học được các mẫu hành vi trừu tượng và phức tạp mà con người khó có thể định nghĩa bằng các quy tắc tĩnh. Đặc biệt, với sự bùng nổ của dữ liệu lớn (Big Data) trong môi trường

mạng, các kiến trúc mạng nơ-ron sâu chứng minh được hiệu năng xử lý vượt trội đối với các dữ liệu phi tuyến tính và nhiều chiều, trở thành hướng đi tiềm năng nhất để nâng cao độ chính xác trong việc phát hiện xâm nhập mạng.

1.4. Kết luận chương 1

Chương 1 đã trình bày một cái nhìn tổng quan và hệ thống về bài toán phát hiện xâm nhập mạng. Các khái niệm cốt lõi về Hệ thống Phát hiện và Ngăn chặn Xâm nhập (IDPS), các kiến trúc triển khai (NIDS, HIDS) cũng như cơ chế hoạt động của hai phương pháp phát hiện chính là Dựa trên Chữ ký và Dựa trên Bất thường đã được phân tích chi tiết.

Qua đó, chúng ta nhận thấy rằng mặc dù các phương pháp truyền thống đã đóng góp lớn vào an ninh mạng trong nhiều thập kỷ, nhưng chúng đang dần bộc lộ những hạn chế chí mạng trước sự tinh vi của các cuộc tấn công hiện đại, đặc biệt là sự bất lực trước các tấn công chưa biết (Zero-day) và tỷ lệ báo động giả cao trong các phương pháp thống kê cũ.

Những thách thức này đặt ra nhu cầu cấp thiết phải ứng dụng các công nghệ tiên tiến hơn. Trong đó, Trí tuệ nhân tạo, cụ thể là Học sâu (Deep Learning), với khả năng tự động trích xuất đặc trưng và xử lý dữ liệu phức tạp, được xác định là hướng đi tiềm năng nhất để khắc phục các nhược điểm trên.

Những thách thức này đặt ra nhu cầu cấp thiết phải ứng dụng các công nghệ tiên tiến hơn. Trong đó, trí tuệ nhân tạo, cụ thể là Học sâu (Deep Learning), với khả năng tự động trích xuất đặc trưng và xử lý dữ liệu phức tạp, được xác định là hướng đi tiềm năng nhất để khắc phục các nhược điểm trên.

CHƯƠNG 2. MÔ HÌNH HỌC SÂU VÀ BỘ DỮ LIỆU CIC-IDS2017

2.1. Các mô hình học sâu nền tảng

2.1.1. Mạng Perceptron đa lớp

Mạng Perceptron đa lớp (Multi-Layer Perceptron - MLP) là một trong những kiến trúc mạng nơ-ron nhân tạo truyền thẳng (Feed-forward Artificial Neural Network) cơ bản và phổ biến nhất. MLP ra đời nhằm khắc phục nhược điểm của mạng Perceptron đơn lớp cổ điển trong việc giải quyết các bài toán không phân tách tuyến tính (non-linearly separable).

a. Kiến trúc tổng quát

Một mạng MLP tiêu chuẩn được cấu thành từ ít nhất ba lớp nơ-ron xếp chồng lên nhau:

- **Lớp đầu vào:** Bao gồm các nơ-ron tiếp nhận dữ liệu thô từ bên ngoài (ví dụ: vector đặc trưng của gói tin mạng). Lớp này không thực hiện tính toán mà chỉ có nhiệm vụ truyền tín hiệu sang lớp tiếp theo. Số lượng nơ-ron tại lớp này tương ứng với số chiều của dữ liệu đầu vào.
- **Các lớp ẩn:** Nằm giữa lớp đầu vào và lớp đầu ra. Đây là nơi thực hiện các tính toán chính của mạng nhằm trích xuất các đặc trưng tiềm ẩn từ dữ liệu. Một mạng MLP có thể có một hoặc nhiều lớp ẩn. Các nơ-ron tại đây sử dụng các hàm kích hoạt phi tuyến tính để mô hình hóa các mối quan hệ phức tạp.
- **Lớp đầu ra:** Bao gồm các nơ-ron đưa ra kết quả dự đoán cuối cùng của mạng (ví dụ: xác suất gói tin là tấn công hay bình thường).

b. Cơ chế hoạt động của một Nơ-ron

Trong mô hình MLP, ngoại trừ lớp đầu vào, mỗi nơ-ron đóng vai trò là một đơn vị xử lý thông tin độc lập. Quy trình xác định giá trị đầu ra của một nơ-ron được thực hiện qua ba bước tuần tự:

- **Tổng hợp tín hiệu:** Đầu tiên, nơ-ron tiếp nhận các giá trị đầu ra từ tất cả các nơ-ron thuộc lớp liền trước. Mỗi giá trị này được nhân với một trọng số liên kết (weight) tương ứng để thể hiện mức độ quan trọng của kết nối đó đối với nơ-ron hiện tại.
- **Điều chỉnh ngưỡng:** Tổng các giá trị đã trọng số hóa ở trên sẽ được cộng thêm một tham số gọi là độ lệch (bias). Giá trị này giúp tịnh tiến và điều chỉnh ngưỡng kích hoạt của nơ-ron một cách linh hoạt hơn.

- Kích hoạt phi tuyến: Cuối cùng, kết quả tổng hợp sẽ được đưa qua một hàm kích hoạt phi tuyến như Sigmoid, hoặc ReLU. Hàm này có nhiệm vụ biến đổi dữ liệu để tạo ra giá trị đầu ra chính thức cho nơ-ron tại lớp hiện tại, giúp mạng nơ-ron có khả năng học và giải quyết các bài toán phức tạp.

c. Quá trình huấn luyện

Quá trình huấn luyện mạng MLP vận hành dựa trên sự kết hợp giữa thuật toán lan truyền ngược và phương pháp tối ưu hóa hạ đạo hàm. Quy trình này diễn ra qua hai giai đoạn lặp đi lặp lại liên tục:

Đầu tiên là giai đoạn Lan truyền thẳng (Feed-forward), trong đó dữ liệu bắt đầu từ lớp đầu vào, đi qua các lớp ẩn để xử lý và cuối cùng tạo ra kết quả dự đoán tại lớp đầu ra. Tiếp theo là giai đoạn Lan truyền ngược (Back-propagation), hệ thống sẽ tính toán mức độ sai số giữa kết quả dự đoán so với nhãn thực tế. Sai số này sau đó được truyền ngược lại từ lớp đầu ra về các lớp trước đó. Mục đích của việc này là để tinh chỉnh lại các tham số trọng số và độ lệch, nhằm giảm thiểu sai số xuống mức thấp nhất trong các lần lặp tiếp theo.

Về bản chất, MLP đóng vai trò là kiến trúc nền tảng. Khi quy mô hệ thống được mở rộng với số lượng lớn các lớp ẩn và nơ-ron nhằm giải quyết các bài toán phức tạp hơn, kiến trúc này sẽ phát triển thành khái niệm Mạng nơ-ron sâu (Deep Neural Network - DNN).

2.1.2. Mạng nơ-ron sâu - Deep Neural Network

Mạng nơ-ron sâu (DNN) là một thuật ngữ bao quát dùng để chỉ các mạng nơ-ron nhân tạo có cấu trúc "sâu", tức là sở hữu nhiều lớp ẩn (thường là lớn hơn 2 lớp) giữa lớp đầu vào và lớp đầu ra. Về mặt bản chất kỹ thuật, DNN là sự mở rộng và phát triển từ kiến trúc MLP truyền thống, nhưng với khả năng tính toán và mức độ trừu tượng hóa dữ liệu cao hơn vượt trội.

a. Đặc điểm kiến trúc

Khác với các mạng nơ-ron nông chỉ có khả năng mô phỏng các hàm số đơn giản, kiến trúc của DNN được thiết kế để giải quyết các bài toán có không gian đặc trưng phức tạp và số chiều lớn.

- Tính kết nối đầy đủ: Trong kiến trúc DNN tiêu chuẩn, các nơ-ron giữa hai lớp liên kế thường được kết nối đầy đủ với nhau (Dense layers). Điều này cho phép mạng tận dụng tối đa thông tin từ toàn bộ các đặc trưng đầu vào.

- Cấu trúc hình tháp: Trong thực tế thiết kế (như mô hình đề xuất trong bài báo cáo), số lượng nơ-ron thường giảm dần qua các lớp (ví dụ: $512 \rightarrow 256 \rightarrow 128$). Cấu trúc này giúp mạng cô đọng thông tin, loại bỏ nhiễu và tập trung vào các đặc trưng cốt lõi nhất khi đi sâu vào các lớp trong cùng.

b. Khả năng học biểu diễn

Ưu điểm lớn nhất của DNN so với các thuật toán máy học truyền thống (như SVM, Decision Tree) là khả năng học phân cấp đặc trưng:

- Các lớp ẩn đầu tiên: Có xu hướng học các đặc trưng cấp thấp, đơn giản và rời rạc từ dữ liệu thô, ví dụ: các byte đơn lẻ hoặc các trường thông tin cơ bản trong gói tin.
- Các lớp ẩn sâu hơn: Tổng hợp các đặc trưng cấp thấp để hình thành nên các đặc trưng trừu tượng và phức tạp hơn, ví dụ: chuỗi hành vi, mẫu lưu lượng bất thường.

Quá trình này diễn ra hoàn toàn tự động trong quá trình huấn luyện mà không cần sự can thiệp thủ công của con người để chọn lọc đặc trưng (Feature Engineering), giúp DNN đặc biệt hiệu quả trong lĩnh vực phát hiện xâm nhập mạng, nơi dữ liệu thường xuyên thay đổi và có tính biến động cao.

2.1.3. Mạng nơ-ron tích chập một chiều

a. Khái niệm và tổng quan

Mạng nơ-ron tích chập (Convolutional Neural Network - CNN) ban đầu được phát triển và gặt hái thành công vang dội với khả năng xử lý dữ liệu hình ảnh hai chiều (2D). Tuy nhiên, đối với các dữ liệu dạng chuỗi, tín hiệu thời gian hoặc các vector đặc trưng dạng bảng như trong bài toán phát hiện xâm nhập mạng, kiến trúc mạng nơ-ron tích chập 1 chiều (1D-CNN) đã được chứng minh là một biến thể tối ưu và hiệu quả.

Khác với 2D-CNN trượt bộ lọc (kernel) trên cả hai chiều không gian: cao và rộng, 1D-CNN sử dụng bộ lọc trượt dọc theo một chiều duy nhất: chiều thời gian hoặc chiều đặc trưng. Trong bài toán phát hiện xâm nhập, đầu vào là một vector đặc trưng 1 chiều. Bộ lọc của 1D-CNN sẽ quét qua vector này để tìm kiếm các mối tương quan giữa các đặc trưng lân cận nhau. Công thức toán học của phép tích chập 1D tại vị trí k được biểu diễn như sau:

$$y_k = f\left(\sum_{i=0}^{M-1} w_i \cdot x_{k+i} + b\right)$$

Trong đó:

- x : Vector đầu vào.
- w : Bộ lọc (Kernel) có kích thước M .
- b : Độ lệch (Bias).
- f : Hàm kích hoạt phi tuyến.

b. Các thành phần chính trong kiến trúc 1D-CNN

Kiến trúc của mô hình 1D-CNN đề xuất được thiết kế dựa trên quy trình xử lý thông tin tuần tự, chuyển đổi dữ liệu thô từ không gian đặc trưng ban đầu sang không gian quyết định. Cấu trúc tổng thể của mạng được chia ba loại lớp chính:

- Lớp tích chập (Convolutional Layer): Đóng vai trò là bộ trích xuất đặc trưng. Các bộ lọc học được sẽ tự động phát hiện các mẫu hình đặc thù của tín hiệu mà không cần chuyên gia định nghĩa trước.
- Lớp gộp (Pooling Layer): Thường sử dụng Max Pooling hoặc. Nhiệm vụ của lớp này là giảm chiều dữ liệu, giữ lại các đặc trưng nổi bật nhất và giảm chi phí tính toán, đồng thời giúp mô hình chống lại hiện tượng quá khớp.
- Lớp kết nối đầy đủ: Sau khi đi qua các lớp tích chập và gộp, dữ liệu được duỗi phẳng (Flatten) thành vector 1 chiều và đưa vào mạng MLP truyền thống để thực hiện việc phân loại cuối cùng (như phân loại nhị phân hoặc đa lớp).

2.1.4. Mối quan hệ giữa MLP, DNN và CNN

Mặc dù MLP, DNN và CNN đều là các kiến trúc thuộc lĩnh vực Học sâu (Deep Learning), giữa chúng có mối quan hệ kế thừa và những đặc điểm cấu trúc riêng biệt quyết định phạm vi ứng dụng của từng mô hình.

a. Mối quan hệ về mặt tiến hóa

Có thể hình dung mối quan hệ giữa ba kiến trúc này theo tiến trình phát triển như sau:

- MLP là kiến trúc nền tảng cơ bản nhất. Nó định nghĩa cơ chế lan truyền thẳng và lan truyền ngược để cập nhật trọng số.

- DNN thực chất là một phiên bản mở rộng của MLP về chiều sâu. Khi số lượng lớp ẩn của MLP tăng lên đáng kể (Deep), nó được gọi là DNN. Do đó, về mặt kỹ thuật, mọi mạng MLP nhiều lớp đều là DNN, nhưng thuật ngữ DNN thường dùng để nhấn mạnh vào độ phức tạp và khả năng trừu tượng hóa.
- CNN là một biến thể tối ưu hóa của DNN. Thay vì kết nối tất cả các nơ-ron với nhau (gây bùng nổ số lượng tham số), CNN sử dụng cơ chế tích chập và chia sẻ trọng số để xử lý dữ liệu có cấu trúc topo như ảnh hiệu quả hơn.

b. So sánh đặc tính kỹ thuật

Sự khác biệt cốt lõi giữa DNN (dựa trên MLP) và CNN nằm ở cách thức kết nối giữa các nơ-ron:

Bảng 2-1. Bảng so sánh mô hình mạng nơ-ron sâu với mạng tích chập

Đặc điểm	Mạng nơ-ron sâu (DNN/MLP)	Mạng tích chập (CNN)
Kiểu kết nối	Kết nối đầy đủ: Mọi nơ-ron lớp trước nối với mọi nơ-ron lớp sau.	Kết nối cục bộ: Nơ-ron chỉ kết nối với một vùng nhỏ cục bộ của lớp trước thông qua bộ lọc.
Số lượng tham số	Rất lớn: Tăng cấp số nhân theo kích thước đầu vào và số lớp.	Ít hơn: Nhờ cơ chế chia sẻ trọng số (Parameter Sharing), cùng một bộ lọc quét qua toàn bộ dữ liệu.
Khả năng học	Tốt trong việc học các mối quan hệ toàn cục (Global patterns).	Tốt trong việc học các đặc trưng bất biến dịch chuyển và mẫu hình cục bộ.
Ứng dụng chính	Dữ liệu dạng bảng, dữ liệu phi cấu trúc không gian.	Dữ liệu ảnh (2D), dữ liệu chuỗi/tín hiệu (1D).

c. Định hướng sử dụng trong nghiên cứu

Trong phạm vi bài báo cáo này, cả hai kiến trúc đều được nghiên cứu và triển khai:

- DNN: Được sử dụng như một mô hình cơ sở (Baseline) mạnh mẽ để thiết lập chuẩn đánh giá trên bộ dữ liệu CIC-IDS-2017, tận dụng khả năng học từ các đặc trưng thống kê của gói tin.

- 1D-CNN: Được đề xuất để khai thác các mối tương quan không gian giữa các đặc trưng lân cận trong vector dữ liệu mạng, nhằm tìm kiếm sự cải thiện về hiệu năng phát hiện tấn công.

2.2. Các thành phần kỹ thuật cốt lõi và phương pháp đánh giá

2.2.1. Hàm kích hoạt

Hàm kích hoạt đóng vai trò đưa tính phi tuyến vào mạng nơ-ron, cho phép mô hình học được các ranh giới quyết định phức tạp. Trong phạm vi nghiên cứu này, ba hàm kích hoạt phổ biến nhất được sử dụng bao gồm: ReLU, Sigmoid và Softmax.

a. Hàm ReLU - Rectified Linear Unit

ReLU là hàm kích hoạt được sử dụng phổ biến nhất cho các lớp ẩn trong các mô hình học sâu hiện đại. ReLU giúp khắc phục vấn đề biến mất đạo hàm, điều này giúp gradient truyền ngược qua nhiều lớp mà không bị suy giảm về 0. ReLU chỉ bao gồm các phép so sánh đơn giản, giúp giảm thời gian huấn luyện mô hình

$$f(x) = \max(0, x)$$

Hàm trả về giá trị 0 nếu đầu vào $x < 0$ và giữ nguyên giá trị x nếu $x \geq 0$.

b. Hàm Sigmoid

Sigmoid là hàm kích hoạt có dạng hình chữ S, thường được sử dụng tại lớp đầu ra cho các bài toán phân loại nhị phân.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Hàm Sigmoid chuyển đổi mọi giá trị thực đầu vào thành một giá trị nằm trong khoảng $(0, 1)$ để biểu thị xác suất dự đoán. Ví dụ:

- Nếu $\sigma \geq 0.5 \rightarrow$ Dự đoán là Tấn công.
- Nếu $\sigma < 0.5 \rightarrow$ Dự đoán là Bình thường.

c. Hàm Softmax

Softmax là hàm kích hoạt được sử dụng tại lớp đầu ra cho các bài toán phân loại đa lớp. Softmax đảm bảo tổng xác suất của các lớp đầu ra luôn bằng 1.

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Trong đó K là số lượng lớp (ví dụ: DoS, Web Attack, Botnet...).

2.2.2. Hàm mất mát

Hàm mất mát (Loss Function), hay còn gọi là hàm chi phí, là thước đo định lượng sai số giữa giá trị dự đoán của mô hình ($\hat{y}$) và nhãn thực tế của dữ liệu (y). Mục tiêu của quá trình huấn luyện mạng nơ-ron là cực tiểu hóa giá trị của hàm mất mát này thông qua việc điều chỉnh các trọng số mạng.

Tùy thuộc vào bản chất của bài toán phân loại: Nhị phân hay đa lớp, nghiên cứu sử dụng hai hàm mất mát sau:

a. Binary Cross-Entropy

Áp dụng cho kịch bản phân loại nhị phân (Bình thường vs. Tấn công). Hàm phạt nặng các dự đoán sai lệch với độ tin cậy cao. Công thức cho một mẫu dữ liệu:

$$L(y_i, (\hat{y}_i)) = [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

Trong đó:

- y_i : Nhãn thực tế (0 hoặc 1).
- $\hat{y}_i$: Xác suất dự đoán đầu ra từ hàm Sigmoid.

Cơ chế hoạt động: Nếu nhãn thực là 1 ($y = 1$): Nếu $\hat{y}_i$ càng gần 1 (dự đoán đúng), Loss càng gần 0. Ngược lại, nếu $\hat{y}_i$ gần 0 (dự đoán sai), Loss sẽ tăng vọt lên vô cùng. Điều này phạt nặng các dự đoán sai lầm tự tin, buộc mô hình phải học cách phân tách rõ ràng hai lớp dữ liệu.

b. Categorical Cross-Entropy

Áp dụng cho kịch bản phân loại đa lớp (Benign, DoS, PortScan...). Hàm đo lường sự khác biệt phân phối xác suất. Công thức:

$$L(y, \hat{y}) = - \sum_{j=1}^c y_j \cdot \log(\hat{y}_j)$$

Trong đó:

- c : Tổng số lượng lớp cần phân loại.
- y_j : Nhãn thực tế dưới dạng one-hot encoding (ví dụ: lớp thứ 2 là $[0, 1, 0, \dots]$).
- $\hat{y}_j$: Xác suất dự đoán cho lớp j từ hàm Softmax.

Cơ chế hoạt động: Hàm này đo lường sự khác biệt phân phối giữa vector xác suất dự đoán và vector nhãn thực tế. Giá trị Loss càng thấp chứng tỏ phân phối xác suất dự đoán của mô hình càng khớp với thực tế (ví dụ: mô hình tự tin dự đoán 99% là DoS cho một mẫu DoS thực sự).

2.2.3. Thuật toán tối ưu hóa và kỹ thuật chống quá khớp

a. Thuật toán tối ưu hóa

Trong nghiên cứu này, thuật toán Adam được lựa chọn làm giải pháp tối ưu hóa chính cho cả hai kịch bản mô hình. Adam là một thuật toán tối ưu thích nghi (adaptive), được xem là sự kết hợp ưu điểm của hai phương pháp mở rộng từ Gradient Descent. Adam cho phép tự động điều chỉnh tốc độ học (learning rate) cho từng tham số dựa trên các ước lượng moment bậc nhất và bậc hai của gradient. Adam giúp mô hình hội tụ nhanh trên dữ liệu lớn và nhiễu.

b. Kỹ thuật chống quá khớp

Để giải quyết vấn đề học quá khớp và đảm bảo tính tổng quát hóa, nghiên cứu áp dụng đồng thời ba kỹ thuật: Dropout, Batch Normalization và Early Stopping.

- Dropout được áp dụng để tắt ngẫu nhiên một tỷ lệ nơ-ron trong quá trình huấn luyện. Điều này giúp ngăn các nơ-ron phụ thuộc quá mức vào nhau, buộc mỗi nơ-ron phải tự học các đặc trưng mạnh mẽ một cách độc lập.
- Batch Normalization chuẩn hóa đầu vào của từng lớp ẩn để đưa dữ liệu về phân phối chuẩn, giúp giải quyết hiện tượng biến thiên hiệp biến nội bộ và ổn định quá trình huấn luyện.
- Early Stopping là chiến lược kiểm soát thời gian huấn luyện dựa trên hiệu năng thực tế. Thay vì cố định số vòng lặp, hệ thống sẽ liên tục giám sát giá trị hàm mất mát trên tập kiểm định sau mỗi epoch, nếu không giảm trong một số epoch liên tiếp thì quá trình huấn luyện sẽ dừng lại.

2.2.4. Các độ đo đánh giá hiệu năng

Để đánh giá khách quan hiệu quả của mô hình phát hiện xâm nhập, nghiên cứu sử dụng hệ thống các chỉ số định lượng tiêu chuẩn dựa trên ma trận nhầm lẫn.

a. Ma trận nhầm lẫn

Ma trận nhầm lẫn là bảng đối chiếu giữa kết quả dự đoán của mô hình và nhãn thực tế của dữ liệu. Đối với bài toán phân loại nhị phân, ma trận này bao gồm 4 thành phần cơ bản:

- True Positive (TP - Dương tính thật): Số mẫu tấn công được phân loại đúng là tấn công.

- True Negative (TN - Âm tính thật): Số mẫu bình thường được phân loại đúng là bình thường.
- False Positive (FP - Dương tính giả): Số mẫu bình thường bị nhận diện nhầm là tấn công (Báo động giả).
- False Negative (FN - Âm tính giả): Số mẫu tấn công bị nhận diện nhầm là bình thường (Bỏ sót tấn công).

b. Các chỉ số đánh giá cơ bản

Dựa trên 4 thông số của ma trận nhầm lẫn, các chỉ số sau được tính toán để đo lường hiệu năng:

- Độ chính xác (Accuracy): Tỷ lệ tổng thể các dự đoán đúng trên tổng số mẫu kiểm thử. Accuracy có thể gây hiểu nhầm nếu tập dữ liệu mất cân bằng (imbalanced data), do đó cần xem xét kết hợp các chỉ số khác.
- Độ chính xác dự báo (Precision): Phản ánh độ tin cậy của các cảnh báo. Trong số các mẫu được báo là tấn công, bao nhiêu % là tấn công thật. Precision thấp đồng nghĩa với việc hệ thống đưa ra nhiều cảnh báo giả, gây lãng phí tài nguyên xử lý của quản trị viên.
- Độ phủ / Độ nhạy (Recall): Phản ánh khả năng bao quát các mối đe dọa. Trong tổng số các cuộc tấn công thực tế, mô hình phát hiện được bao nhiêu %. Trong an toàn thông tin, Recall là chỉ số quan trọng nhất. Recall thấp đồng nghĩa với việc hệ thống bỏ lọt nhiều tấn công, gây nguy hiểm cho hệ thống.
- Điểm F1 (F1-Score): Là trung bình điều hòa giữa Precision và Recall, dùng để đánh giá sự cân bằng của mô hình.

c. Đường cong ROC và chỉ số AUC

Đường cong ROC (Receiver Operating Characteristic): Biểu diễn mối quan hệ đánh đổi giữa tỷ lệ dương tính thật và tỷ lệ dương tính giả tại các ngưỡng quyết định khác nhau.

Chỉ số AUC (Area Under the Curve): Diện tích nằm dưới đường cong ROC. Một mô hình phân loại hoàn hảo $AUC = 1$, trong khi mô hình đoán ngẫu nhiên có $AUC = 0.5$.

2.3. Bộ dữ liệu CIC-IDS-2017

2.3.1. Giới thiệu tổng quan và lý do lựa chọn

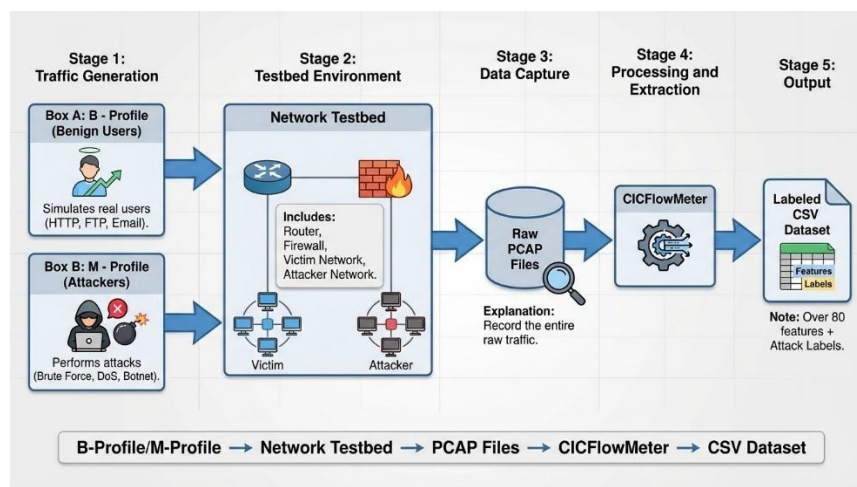
a. Giới thiệu tổng quan

Bộ dữ liệu CIC-IDS-2017 (Canadian Institute for Cybersecurity Intrusion Detection System 2017) được xây dựng bởi Viện An ninh mạng Canada thuộc Đại học New Brunswick. Đây là bộ dữ liệu được thiết kế nhằm khắc phục các hạn chế của các bộ dữ liệu cũ, hướng tới việc tạo ra một tập dữ liệu chuẩn mực, tin cậy và phản ánh chính xác các đặc điểm của lưu lượng mạng hiện đại.

Điểm đặc biệt của CIC-IDS-2017 là việc áp dụng hệ thống hồ sơ hành vi người dùng (B-Profiles) để sinh lưu lượng nền. Thay vì tạo gói tin ngẫu nhiên, nhóm tác giả đã trừu tượng hóa các tương tác người dùng thực tế thông qua các giao thức chuẩn (HTTP, HTTPS, SSH, SMTP...), giúp tái hiện chính xác quy luật phân bố lưu lượng của một môi trường mạng doanh nghiệp thực tế. Quá trình thu thập dữ liệu được thực hiện trong vòng 05 ngày (từ thứ Hai, ngày 03/07/2017 đến thứ Sáu, ngày 07/07/2017) trên một kiến trúc mạng thử nghiệm hoàn chỉnh gồm hai thành phần:

- Mạng Nạn nhân: Bao gồm các máy trạm và máy chủ (được cài đặt các hệ điều hành phổ biến như Windows, Ubuntu, Mac OS) đóng vai trò là mục tiêu và thực hiện các hoạt động mạng thông thường.
- Mạng Tấn công: Là một mạng tách biệt hoàn toàn, chứa các máy tính thực hiện việc rà quét và triển khai các kịch bản tấn công vào mạng nạn nhân thông qua internet công cộng.

Quy trình tổng thể từ việc sinh dữ liệu đến trích xuất đặc trưng được mô tả chi tiết trong hình dưới đây:



Hình 2-1 Quy trình xây dựng và xử lý bộ dữ liệu CIC-IDS-2017

Như được thể hiện trong hình 2.1, kết quả thu được bao gồm toàn bộ lưu lượng mạng dưới dạng tệp bắt gói tin (PCAP). Để phục vụ cho việc huấn luyện các mô hình học máy và học sâu, dữ liệu này sau đó được xử lý qua công cụ CICFlowMeter. Công cụ này trích xuất và chuyển đổi các gói tin thô thành các luồng (Flows) và xuất ra định dạng bảng (CSV). Mỗi bản ghi luồng dữ liệu bao gồm 80 đặc trưng thống kê mạng (như thời gian tồn tại của luồng, độ lệch chuẩn độ dài gói tin, số lượng gói tin gửi/nhận...) và được gán nhãn chính xác theo từng loại tấn công hoặc nhãn bình thường.

b. Lý do lựa chọn bộ dữ liệu CIC-IDS-2017

Trong cộng đồng nghiên cứu an toàn thông tin, mặc dù có nhiều bộ dữ liệu kinh điển như KDD Cup 99 hay NSL-KDD, nhưng đề tài quyết định lựa chọn CIC-IDS-2017 dựa trên các luận điểm khoa học sau:

Thứ nhất là tính thời sự, bộ dữ liệu khắc phục được vấn đề "lão hóa" của các mẫu cũ bằng cách cập nhật các hình thái tấn công mới như Botnet, DoS tăng ứng dụng và các khai thác lỗ hổng Web. Thứ hai là chất lượng nhãn dán, dữ liệu được gán nhãn chính xác cho từng luồng, tạo điều kiện lý tưởng cho việc huấn luyện các thuật toán Học máy có giám sát. Cuối cùng và quan trọng nhất là tính phù hợp với mô hình Học sâu, hệ thống đặc trưng dựa trên luồng (Flow-based features) cho phép mô hình phân tích hành vi bất thường hiệu quả ngay cả trên lưu lượng bị mã hóa mà không cần can thiệp giải mã nội dung gói tin.

2.3.2. Mô tả các loại tấn công trong bộ dữ liệu

Bộ dữ liệu CIC-IDS-2017 mô phỏng một môi trường mạng đa dạng với sự hiện diện của 14 loại tấn công khác nhau bên cạnh lưu lượng mạng bình thường (Benign). Các cuộc tấn công này bao gồm từ các kỹ thuật rà quét cơ bản đến các phương thức khai thác lỗ hổng tăng ứng dụng phức tạp, được chia thành các nhóm chính sau:

- Nhóm tấn công từ chối dịch vụ (DoS và DDoS): Chiếm tỷ trọng lớn nhất, mục tiêu là làm cạn kiệt tài nguyên hệ thống. Nhóm này bao gồm các kỹ thuật tấn công HTTP Flood (DoS Hulk, GoldenEye), tấn công chậm (Slowloris, Slowhttptest) và tấn công phân tán (DDoS) .
- Nhóm tấn công dò mật khẩu (Brute Force): Sử dụng các công cụ tự động để thử sai mật khẩu liên tục nhằm chiếm quyền truy cập trái phép thông qua các giao thức truyền tệp (FTP-Patator) và quản trị từ xa (SSH-Patator) .

- Nhóm tấn công ứng dụng Web (Web Attacks): Tập trung khai thác các lỗ hổng trong mã nguồn website, bao gồm tấn công chèn mã độc (XSS), chèn câu lệnh truy vấn (SQL Injection) và dò mật khẩu trang quản trị (Web Brute Force) .
- Nhóm rà quét mạng (PortScan): Thực hiện gửi các gói tin thăm dò đến máy chủ đích để xác định các cổng mở và dịch vụ đang chạy, thường là bước tiền trạm cho một cuộc tấn công lớn hơn.
- Nhóm các mối đe dọa tiên tiến (Advanced Threats): Bao gồm các loại mã độc nguy hiểm và khó phát hiện như mạng máy tính ma (Botnet), phần mềm gián điệp xâm nhập mạng nội bộ (Infiltration) và khai thác lỗ hổng bộ nhớ nghiêm trọng Heartbleed .

2.4. Phân tích khám phá bộ dữ liệu

2.4.1. Phân tích đặc điểm và cấu trúc bộ dữ liệu

a. Cấu trúc tổ chức dữ liệu

Bộ dữ liệu CIC-IDS-2017 sau khi được trích xuất từ các tệp PCAP thông qua công cụ CICFlowMeter được lưu trữ dưới dạng 8 tệp tin CSV (Comma-separated values), tương ứng với các ngày thu thập và các kịch bản tấn công khác nhau.

- Tổng kích thước: Khoảng 2.8 triệu bản ghi.
- Số chiều dữ liệu (Dimensions): Mỗi bản ghi bao gồm 79 cột (đặc trưng).

Bảng 2-2. Danh sách các tệp dữ liệu thành phần của CIC-IDS-2017

Tên tệp tin	Kịch bản tấn công chủ đạo	Số lượng mẫu
Monday-WorkingHours.pcap_ISCX.csv	Benign (Lưu lượng sạch)	529918
Tuesday-WorkingHours.pcap_ISCX.csv	FTP-Patator, SSH-Patator	445909
Wednesday-workingHours.pcap_ISCX.csv	DoS / DDoS, Heartbleed	692703
Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv	Web Attacks (Brute Force, XSS, SQL)	170366

Thursday-WorkingHours-Afternoon-Infiltration.pcap_ISCX.csv	Infiltration	288602
Friday-WorkingHours-Morning.pcap_ISCX.csv	Botnet	191033
Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv	PortScan	286467
Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv	DDoS	225745

b. Không gian đặc trưng

Dựa trên kết quả trích xuất từ công cụ CICFlowMeter, không gian đặc trưng của bộ dữ liệu bao gồm 78 thuộc tính đầu vào và 1 nhãn đầu ra. Các đặc trưng này tập trung mô tả hành vi của luồng dữ liệu (Flow) theo hai hướng: Xuôi chiều (Forward - từ nguồn đến đích) và Ngược chiều (Backward - từ đích về nguồn). Chúng có thể được phân loại thành 5 nhóm chính như sau:

- Nhóm thông kê thời gian (Time & IAT): Đây là nhóm đặc trưng quan trọng nhất để phát hiện các tấn công dựa trên thời gian như DoS hoặc Botnet. Các đặc trưng như thời gian duy trì luồng, khoảng thời gian đến giữa các gói tin (Inter-Arrival Time), thời gian hoạt động/ngủ của luồng .
- Nhóm khối lượng (Volume): Nhóm này mô tả lượng dữ liệu được truyền tải, giúp phát hiện các bất thường về băng thông. Các đặc trưng tiêu biểu: tổng số gói tin, tổng dung lượng (Bytes), và các chỉ số thống kê (Min, Max, Mean, Std) về độ dài gói tin.
- Nhóm cờ hiệu (Flags): Số lượng các cờ TCP (SYN, FIN, RST, ACK...) trong phần tiêu đề, phản ánh trạng thái kết nối .
- Nhóm tốc độ (Rate): Tốc độ truyền tải (Bytes/s, Packets/s) và kích thước cửa sổ truyền .
- Nhãn phân loại Label: Đây là cột mục tiêu (Target variable) chứa thông tin phân loại của luồng dữ liệu. Trong bài toán này, nó sẽ nhận các giá trị như "BENIGN" hoặc tên các loại tấn công ("DoS Hulk", "PortScan", "Bot"...).

2.4.2. Đánh giá chất lượng và mối quan hệ giữa các đặc trưng

a. Đánh giá sơ bộ chất lượng dữ liệu

Thông qua quá trình thống kê mô tả và kiểm tra sơ bộ trên toàn bộ 8 tệp dữ liệu thành phần (gồm hơn 2.8 triệu bản ghi), em đã nhận diện được một số vấn đề kỹ thuật ảnh hưởng trực tiếp đến chất lượng dữ liệu. Việc phát hiện sớm các vấn đề này là cơ sở quan trọng để xây dựng quy trình tiền xử lý tại Chương 3. Các vấn đề chính bao gồm:

- Dữ liệu lỗi và nhiễu: Tồn tại các bản ghi chứa giá trị vô hạn hoặc rỗng (NaN) do lỗi chia cho 0 trong quá trình tính toán thống kê. Bên cạnh đó, xuất hiện dữ liệu trùng lặp có thể gây thiên kiến đánh giá
- Đặc trưng tĩnh không mang thông tin: Đây là vấn đề đáng lưu ý nhất về mặt cấu trúc không gian đặc trưng. Qua phân tích xác định được 8 cột "chết" (như Fwd Avg Bytes/Bulk, Bwd PSH Flags, Bwd URG Flags...) có giá trị bằng 0 ở mọi bản ghi. Các đặc trưng này không mang lại lượng tin cho việc phân loại và cần được loại bỏ để giảm chiều dữ liệu.

Từ đó có thể kết luận rằng, bộ dữ liệu CIC-IDS-2017 mặc dù rất toàn diện nhưng vẫn tồn tại các nhiễu và dư thừa đặc trưng. Do đó, trước khi đưa vào huấn luyện mô hình Học sâu, em sẽ thực hiện loại bỏ toàn bộ các bản ghi lỗi, bản ghi trùng lặp và 8 đặc trưng tĩnh nêu trên. Quy trình này sẽ được trình bày chi tiết trong phần Tiền xử lý dữ liệu ở chương tiếp theo.

b. Phân tích tương quan giữa các đặc trưng

Trước khi đi vào phân tích dữ liệu cụ thể, cần làm rõ các khái niệm thống kê được sử dụng để đánh giá mối quan hệ giữa các đặc trưng:

Hệ số tương quan Pearson (r) : Đây là thước đo thống kê phản ánh mức độ quan hệ tuyến tính giữa hai biến số. Giá trị của (r) nằm trong khoảng $[-1, 1]$:

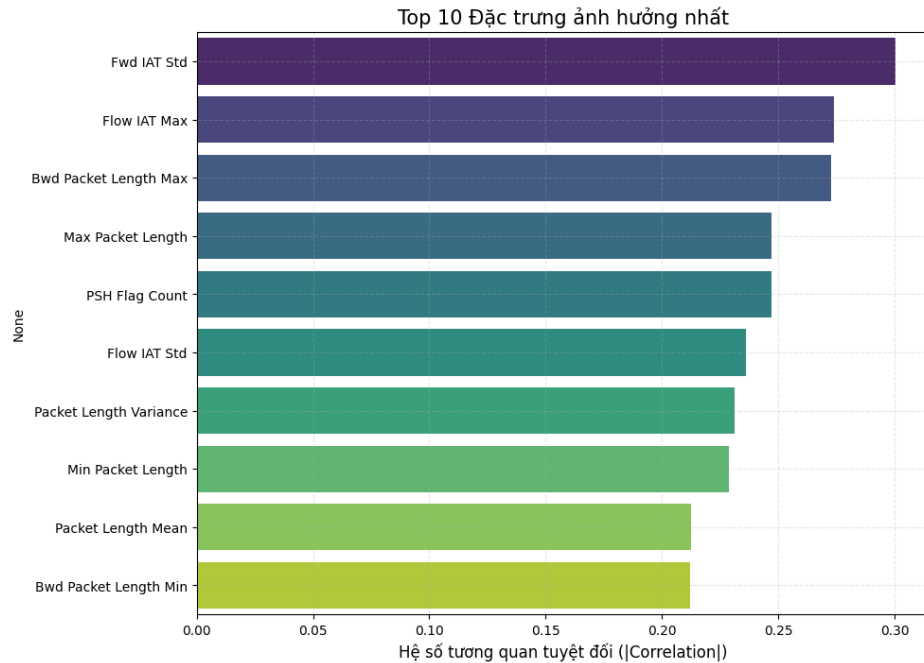
Hiện tượng đa cộng tuyến: Trong bài toán học máy, đa cộng tuyến xảy ra khi hai hoặc nhiều biến độc lập (đặc trưng đầu vào) có mối tương quan rất mạnh với nhau. Điều này dẫn đến sự dư thừa thông tin, làm tăng chi phí tính toán không cần thiết và có thể gây nhiễu, khiến mô hình khó xác định được trọng số chính xác cho từng đặc trưng.

Để phát hiện hiện tượng này trong bộ dữ liệu CIC-IDS-2017, nghiên cứu sử dụng biểu đồ nhiệt để trực quan hóa ma trận tương quan giữa 78 đặc trưng. Trong biểu đồ dưới đây, màu càng đậm thể hiện hệ số tương quan càng cao (tiệm cận 1.0).

số tương quan cao (ví dụ: $r > 0,9$), giúp giảm chiều dữ liệu và tối ưu hóa hiệu suất huấn luyện cho mô hình Học sâu.

c. Đánh giá mức độ quan trọng của đặc trưng đối với Nhãn

Khác với việc đánh giá đa cộng, mục tiêu của bước này là xác định mức độ liên quan của từng đặc trưng đối với biến mục tiêu (*Label*). Hình dưới đây hiển thị Top 10 đặc trưng có hệ số tương quan cao nhất trên tập mẫu phân tầng, phản ánh mức độ ảnh hưởng của chúng đến khả năng phát hiện tấn công.



Hình 2-3.10 đặc trưng có ảnh hưởng lớn nhất đến nhãn

Dựa trên kết quả xếp hạng hệ số tương quan Pearson giữa đặc trưng và nhãn mục tiêu (Hình 2.3), nghiên cứu rút ra các quan sát cốt lõi sau:

- Nhóm đặc trưng chủ đạo: Các đặc trưng thuộc nhóm hành vi/thời gian và kích thước gói tin chiếm vị trí cao trong việc nhận diện tấn công. Điều này phản ánh sự khác biệt giữa hành vi truy cập ngẫu nhiên của con người so với các mẫu tấn công tự động, liên tục và có tải trọng khác biệt của máy móc.
- Vấn đề dư thừa dữ liệu: Ngược lại, thực nghiệm chỉ ra sự tồn tại của một lượng lớn đặc trưng có hệ số tương quan tiệm cận 0. Việc giữ lại các biến này không cải thiện độ chính xác mà còn làm tăng chiều dữ liệu không cần gây lãng phí tài nguyên tính toán.

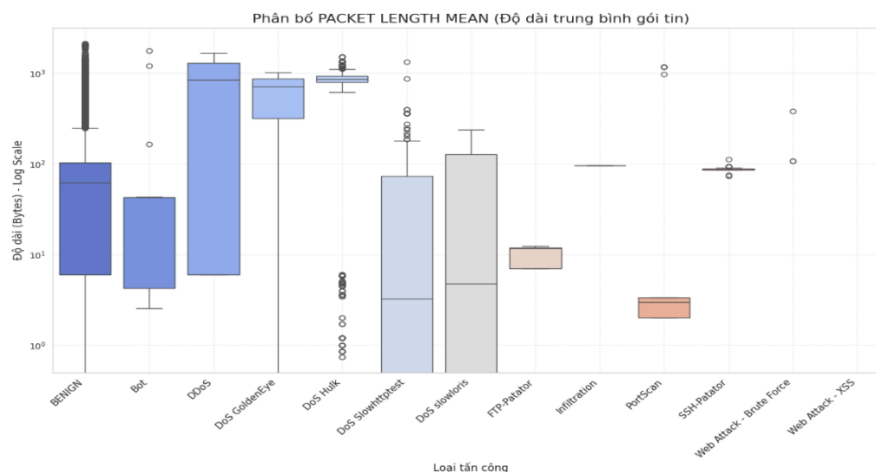
Sự phân hóa rõ rệt này là cơ sở khoa học để nghiên cứu áp dụng chiến lược lựa chọn đặc trưng trong Chương 3. Cụ thể, các đặc trưng có hệ số tương quan thấp dưới ngưỡng 0.01 sẽ bị loại bỏ nhằm nén không gian dữ liệu.

2.4.3. Phân tích hành vi và phân bố dữ liệu

a. Phân tích phân bố theo lớp tấn công

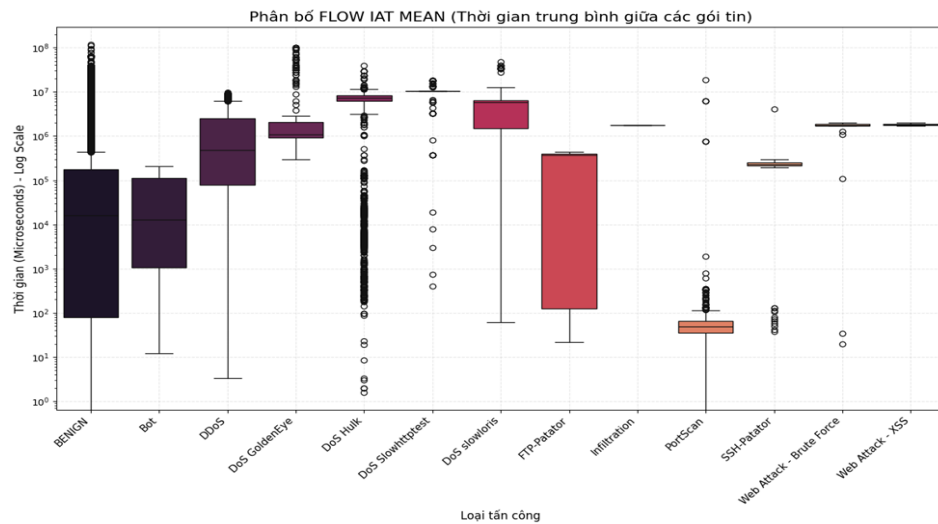
Để hiểu rõ hơn về hành vi của các loại tấn công khác nhau và đánh giá khả năng phân loại của các đặc trưng, nghiên cứu sử dụng biểu đồ hộp để so sánh phân phối dữ liệu giữa nhãn bình thường và các nhóm tấn công khác nhau. Kết quả cho thấy một thách thức lớn trong việc phân loại: Sự chồng lấn giữa hành vi bình thường và tấn công.

- Đặc điểm của tấn công (Tính tự động hóa): Các hộp dữ liệu của nhóm tấn công (như DoS, Botnet) thường có biên độ hẹp và tập trung tại các vùng giá trị cao. Ví dụ, tại Hình 2.4, các tấn công DoS Hulk hay GoldenEye có kích thước gói tin trung bình tập trung hẳn ở vùng trên 1000 Bytes, tách biệt so với lưu lượng thường. Điều này phản ánh tính chất tự động, lặp lại và dập khuôn của công cụ tấn công để tối ưu hóa hiệu quả phá hoại.



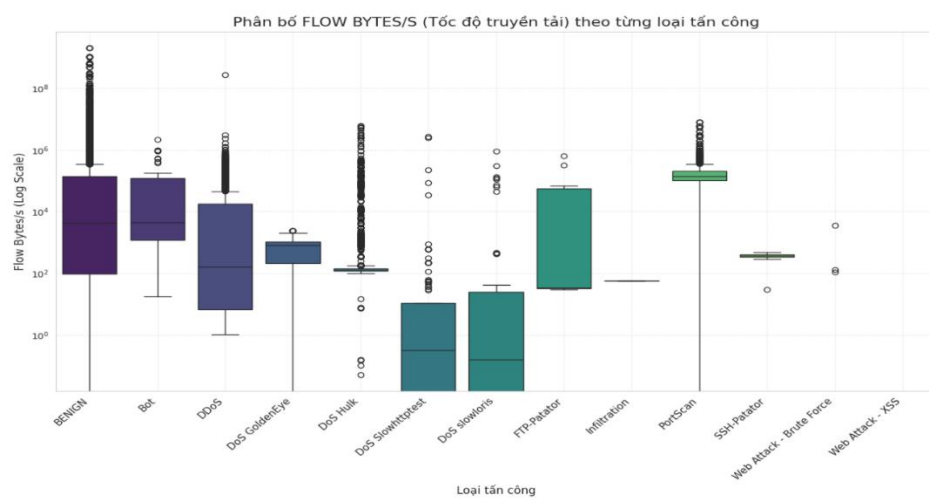
Hình 2-4. Phân bố Độ dài trung bình gói tin theo từng loại tấn công.

- Đặc điểm của lưu lượng sạch (Tính ngẫu nhiên): Ngược lại, lớp Bình thường thể hiện độ phân tán dữ liệu rất rộng. Như quan sát tại Hình 2.5, thời gian giữa các gói tin (IAT) của lớp Benign trải dài trên khoảng giá trị lớn, phản ánh sự đa dạng trong hành vi ngẫu nhiên của người dùng thực tế.



Hình 2-5. Phân bố Thời gian trung bình giữa các gói tin (Flow IAT Mean).

- Thách thức phân loại (Sự chồng lấn): Tuy nhiên, thách thức lớn nhất nằm ở sự chồng lấn dữ liệu (Overlapping). Hình 2.6 cho thấy lớp Benign xuất hiện nhiều giá trị ngoại lai cực lớn về tốc độ truyền tải (Flow Bytes/s), xâm lấn vào vùng giá trị của tấn công DoS. Sự mờ nhạt về ranh giới này là nguyên nhân chính gây ra tỷ lệ báo động giả cao nếu chỉ dùng các bộ lọc đơn giản, khẳng định nhu cầu áp dụng mô hình Học sâu đa chiều.



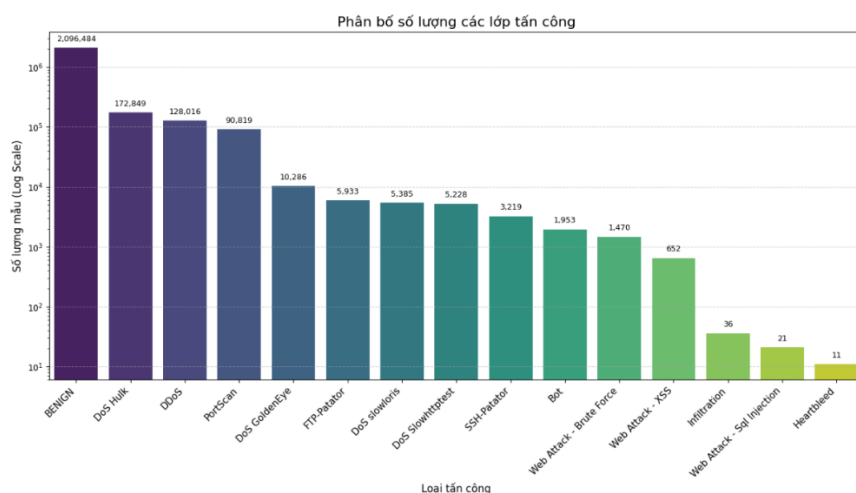
Hình 2-6. Phân bố Flow Bytes/s theo từng loại tấn công (Thang đo Log)

Phân tích cho thấy các đặc trưng đơn lẻ chỉ hiệu quả với một số kiểu tấn công nhất định, trong khi lưu lượng BENIGN có độ biến thiên lớn và nhiều ngoại lai, làm mờ ranh giới giữa hành vi bình thường và tấn công. Điều này khiến các phương pháp dựa trên ngưỡng đơn giản dễ gây cảnh báo giả, qua đó nhấn mạnh sự cần thiết của

các mô hình Học sâu có khả năng khai thác quan hệ phi tuyến và kết hợp đa đặc trưng để phân loại chính xác hơn.

b. Phân tích sự mất cân bằng dữ liệu

Một đặc điểm cố hữu của dữ liệu giám sát mạng thực tế, và cũng là thách thức lớn nhất trong bộ dữ liệu CIC-IDS-2017, là sự phân bố cực kỳ không đồng đều giữa các lớp dữ liệu.



Hình 2-7. Phân bố số lượng các lớp tấn công trong bộ dữ liệu

Thống kê cho thấy lớp Benign chiếm tỷ trọng áp đảo với hơn 80% tổng số mẫu, trong khi các loại tấn công nguy hiểm và tinh vi như Heartbleed hay Infiltration chỉ xuất hiện với tần suất cực thấp.

Sự chênh lệch tỷ lệ lên tới hàng trăm nghìn lần này dẫn đến hai hệ quả kỹ thuật nghiêm trọng: Thứ nhất là hiện tượng Thiên kiến mô hình (Model Bias), khi hàm mất mát bị chi phối hoàn toàn bởi lớp đa số, khiến mô hình có xu hướng "học vẹt" và luôn dự đoán đầu ra là an toàn để tối thiểu hóa sai số. Thứ hai là Nghịch lý về độ chính xác (Accuracy Paradox), nơi hệ thống có thể đạt độ chính xác tổng thể rất cao (ví dụ 99%) nhưng thực chất lại thất bại hoàn toàn trong việc nhận diện các tấn công hiếm gặp – vốn là những mối đe dọa nguy hiểm nhất cần phát hiện.

Để giải quyết triệt để vấn đề này, thay vì sử dụng phương pháp sao chép mẫu đơn giản dễ gây hiện tượng quá khớp, nghiên cứu đề xuất áp dụng kỹ thuật SMOTE (Synthetic Minority Over-sampling Technique). Đây là thuật toán tăng cường dữ liệu hoạt động dựa trên nguyên lý nội suy tuyến tính trong không gian đặc trưng. Quy trình sinh một mẫu mới bao gồm các bước:

- Chọn mẫu: Chọn ngẫu nhiên một mẫu x_i thuộc lớp thiểu số (ví dụ: một mẫu tấn công Heartbleed).

- Tìm láng giềng: Xác định k láng giềng gần nhất của x_i trong không gian đặc trưng (thường chọn $k = 3$)
- Chọn đối tượng nội suy: Chọn ngẫu nhiên một láng giềng x_{zi} trong số k láng giềng đó.
- Sinh mẫu mới: Tạo ra một mẫu nhân tạo x_{new} nằm trên đoạn thẳng nối giữa x_i và x_{zi} theo công thức vector:

$$x_{new} = x_i + \lambda \times (x_{zi} - x_i)$$

Trong đó, λ là một số ngẫu nhiên trong khoảng $(0, 1)$.

2.5. Kết luận chương 2

Chương 2 đã hệ thống hóa các cơ sở lý thuyết và dữ liệu nền tảng quan trọng nhất cho đề tài. Cụ thể:

Về mô hình: Chương đã trình bày chi tiết kiến trúc và cơ chế hoạt động của các mô hình Học sâu tiêu biểu là DNN (Deep Neural Network) và 1D-CNN (1D-Convolutional Neural Network). Trong đó, DNN được lựa chọn làm mô hình cơ sở nhờ khả năng học các đặc trưng phi tuyến phức tạp, còn 1D-CNN được đề xuất như một giải pháp nâng cao để khai thác các mối quan hệ không gian cục bộ trong dữ liệu mạng.

Về kỹ thuật: Các thành phần cốt lõi để xây dựng một mô hình hiệu quả như hàm kích hoạt (ReLU, Softmax), hàm mất mát (Cross-Entropy), thuật toán tối ưu (Adam) và các kỹ thuật chống quá khớp (Dropout, Batch Normalization) đã được phân tích và lựa chọn phù hợp với từng bài toán cụ thể (Nhị phân và Đa lớp).

Về dữ liệu: Thông qua quá trình phân tích khám phá (EDA) trên bộ dữ liệu CIC-IDS-2017, nghiên cứu đã chỉ ra những thách thức lớn cần giải quyết, bao gồm: không gian đặc trưng dư thừa (nhiều đặc trưng tương quan cao hoặc không mang thông tin), dữ liệu nhiễu và đặc biệt là sự mất cân bằng nghiêm trọng giữa các lớp tấn công.

Những phân tích này là tiền đề then chốt để xây dựng quy trình Tiền xử lý dữ liệu (làm sạch, lựa chọn đặc trưng, cân bằng mẫu với SMOTE) và thiết kế Kịch bản thực nghiệm chi tiết sẽ được trình bày trong Chương 3.

CHƯƠNG 3. TRIỂN KHAI THỰC NGHIỆM

3.1. Môi trường và quy trình thực nghiệm

3.1.1. Môi trường thực nghiệm

Để đảm bảo khả năng tính toán cho các mô hình Học sâu với khối lượng dữ liệu lớn, nhóm nghiên cứu đã triển khai thực nghiệm trên nền tảng đám mây Google Colaboratory (Colab). Đây là môi trường cung cấp tài nguyên phần cứng hiệu năng cao, cho phép tăng tốc quy trình huấn luyện và đánh giá mô hình. Cấu hình phần cứng cụ thể bao gồm GPU NVIDIA Tesla T4 (với 15GB VRAM) hỗ trợ tính toán song song ma trận tốc độ cao, kết hợp với RAM hệ thống 12GB và CPU Intel Xeon @ 2.20GHz. Về mặt phần mềm, hệ thống được xây dựng trên ngôn ngữ Python 3.10, sử dụng thư viện TensorFlow 2.x và Keras làm nền tảng học sâu, kết hợp với các thư viện xử lý dữ liệu tiêu chuẩn như Pandas, NumPy và Scikit-learn.

3.1.2. Quy trình tổng thể

Dựa trên kết quả phân tích dữ liệu ở Chương 2, quy trình xây dựng hệ thống được thiết kế gồm 4 giai đoạn nối tiếp nhau :

- Giai đoạn 1 - Tiền xử lý (Preprocessing): Tập trung làm sạch dữ liệu thô, xử lý giá trị lỗi và chuẩn hóa định dạng.
- Giai đoạn 2 - Tối ưu hóa (Optimization): Lựa chọn đặc trưng quan trọng để giảm chiều dữ liệu và xử lý vấn đề mất cân bằng mẫu (SMOTE/Undersampling) .
- Giai đoạn 3 - Huấn luyện (Training): Triển khai huấn luyện các mô hình DNN và 1D-CNN theo hai kịch bản: Phân loại Nhị phân và Phân loại Đa lớp.
- Giai đoạn 4 - Đánh giá (Evaluation): Kiểm định mô hình trên tập dữ liệu độc lập (Test set) sử dụng bộ chỉ số tiêu chuẩn.

3.2. Tiền xử lý dữ liệu chung

Trong phần này, sẽ trình bày chi tiết các bước kỹ thuật để chuyển đổi bộ dữ liệu thô CIC-IDS2017 thành dạng chuẩn, sẵn sàng cho việc huấn luyện mô hình.

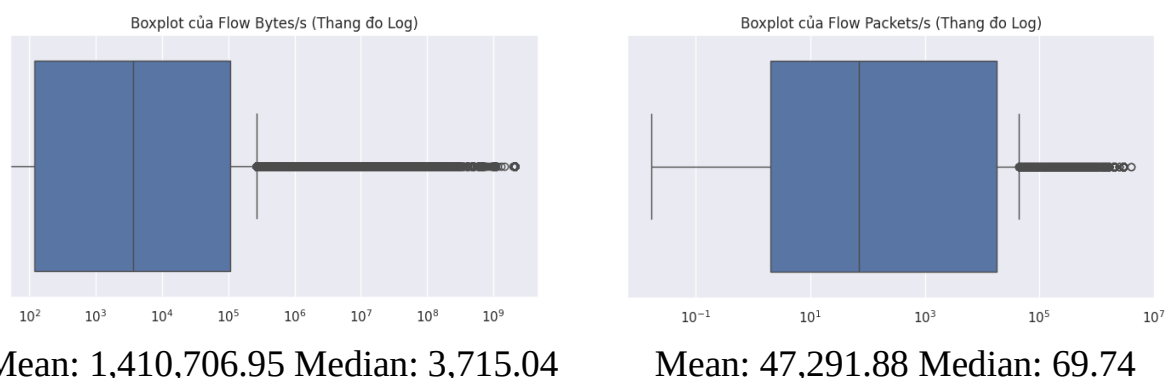
3.2.1. Tích hợp và làm sạch dữ liệu sơ bộ

Bộ dữ liệu gốc bao gồm 8 tệp tin CSV riêng lẻ tương ứng với các ngày thu thập khác nhau, được tích hợp lại thành một bảng dữ liệu duy nhất với kích thước 2,830,743 dòng và 79 cột. Quá trình kiểm tra chất lượng dữ liệu sau khi tích hợp đã phát hiện và xử lý hai vấn đề chính.

Thứ nhất là dữ liệu trùng lặp, hệ thống đã loại bỏ 308381 bản ghi trùng lặp hoàn toàn (chiếm khoảng 11%) để tránh hiện tượng thiên kiến khi đánh giá mô hình. Sau khi xóa kích thước bộ dữ liệu là 2522362 dòng, 79 cột.

Thứ hai là các giá trị lỗi số học (NaN và Infinity) do đặc thù của việc tính toán các chỉ số mạng (ví dụ: chia cho thời gian bằng 0), bộ dữ liệu xuất hiện các giá trị vô cực (Infinity) và giá trị rỗng (NaN). Các giá trị này cần được chuẩn hóa và loại bỏ để đảm bảo tính toán số học chính xác.

Qua phân tích các giá trị lỗi này nằm ở 2 cột: Flow Bytes/s và Flow Packets/s. Ta sẽ đi phân tích phân bố dữ liệu trên 2 cột này để đưa ra giải pháp thay thế.



Hình 3-1 Biểu đồ hộp phân phối của cột Flow Bytes/s (a) và Flow Packets/s (b)

Nhận xét về hai biểu đồ trên ta thấy: Dữ liệu Bị lệch nặng, con số Mean (Trung bình) lớn hơn Median (Trung vị) tới gần 380 lần là bằng chứng không thể chối cãi rằng dữ liệu bị "lệch phải". Do có lượng lớn các giá trị ngoại lai đã kéo giá trị Mean lên cao

Phân tích này chứng minh rằng Median là một giá trị đại diện "bền bỉ" và chính xác hơn Mean. Do đó, việc lấp đầy (fill) các giá trị NaN bằng Median là lựa chọn hợp lý nhất về mặt thống kê.

Đoạn mã thực hiện làm sạch:

```
cols_to_fill_median = ['Flow Bytes/s', 'Flow Packets/s']  
for col in cols_to_fill_median:  
    median_val = data[col].median()  
    data[col] = data[col].fillna(median_val)
```

3.2.2. Chiến lược tối ưu hóa đặc trưng

a. Loại bỏ các cột chết

Qua phân tích thống kê, hệ thống phát hiện có 8 cột đặc trưng có phương sai bằng 0, chỉ chứa một giá trị duy nhất trong toàn bộ tập dữ liệu. Các cột này không mang lại thông tin phân loại và cần được loại bỏ. Đoạn mã xử lý:

```
# Xác định các cột có độ lệch chuẩn (std) bằng 0
std_dev = df.std(numeric_only=True)
dead_cols = std_dev[std_dev == 0].index.tolist()

# Loại bỏ khỏi bộ dữ liệu
df_clean = df.drop(columns=dead_cols)
```

Kết quả là bộ dữ liệu còn 71 cột.

b. Khử đa cộng tuyến - Loại bỏ đặc trưng trùng lặp

Nghiên cứu tính toán ma trận tương quan Pearson giữa tất cả các cặp đặc trưng. Nếu hệ số tương quan giữa hai đặc trưng > 0.95 chúng được coi là trùng lặp thông tin và một trong hai sẽ bị loại bỏ.

Đoạn mã thực thi Giai đoạn 1:

```
# 1. Tính ma trận tương quan trên toàn bộ dữ liệu (bỏ cột Label)
X = df.drop(columns=['Label'])
corr_matrix = X.corr(numeric_only=True).abs()

# 2. Tìm các cặp cột trùng lặp (Hệ số tương quan > 0.95)
# Chọn tam giác trên của ma trận để không duyệt trùng
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape),
k=1).astype(bool))

# Lập danh sách cột cần xóa
to_drop_high_corr = [column for column in upper.columns if
any(upper[column] > 0.95)]

# Thực hiện xóa
df.drop(columns=to_drop_high_corr, inplace=True)
```

Kết quả, phát hiện 23 cột trùng lặp thông tin. Sau khi xóa bộ dữ liệu 48 cột

c. Loại nhiễu - Loại bỏ đặc trưng ít quan trọng

Sau khi khử trùng lặp, bộ dữ liệu vẫn còn tồn tại các đặc trưng có độ biến thiên nhưng ít đóng góp vào việc phân loại nhãn tấn công. Những đặc trưng này được xem

là "nhiều". Giải pháp sẽ là tính toán hệ số tương quan giữa từng đặc trưng còn lại với nhãn mục tiêu (Label). Các đặc trưng có hệ số tương quan thấp hơn ngưỡng 0.01 sẽ bị loại bỏ.

Đoạn mã thực thi Giai đoạn 2:

```
# 1. Tính tương quan giữa đặc trưng và Nhãn
corr_with_target=df.drop(columns=['Label']).corrwith(df['Label']
).abs()

# 2. Lọc các cột có tương quan < 0.01
to_drop_low_corr =
corr_with_target[corr_with_target<0.01].index.tolist()

# Thực hiện xóa
df_final = df.drop(columns=to_drop_low_corr)
```

Kết quả, phát hiện 10 cột có tương quan quá thấp. Sau khi xóa các cột này kích thước bộ dữ liệu là 2522362 hàng, 38 cột.

3.2.3. Phân chia tập và chuẩn hóa tập dữ liệu

a. Phân chia tập dữ liệu

Để đảm bảo tính khách quan trong việc đánh giá mô hình và tránh hiện tượng rò rỉ dữ liệu (Data Leakage), tập dữ liệu sau khi tối ưu hóa đặc trưng được chia thành hai phần độc lập:

- Tập huấn luyện (Training Set - 80%): Dùng để huấn luyện mô hình và áp dụng các kỹ thuật cân bằng dữ liệu.
- Tập kiểm thử (Test Set - 20%): Được giữ nguyên vẹn, không áp dụng SMOTE, dùng để đánh giá hiệu năng thực tế của mô hình.

Nghiên cứu sử dụng kỹ thuật lấy mẫu phân tầng thông qua tham số stratify. Kỹ thuật này đảm bảo tỷ lệ các lớp tấn công hiểm (như Heartbleed, Infiltration) trong tập Test tương đồng với tập dữ liệu gốc, tránh trường hợp tập Test không có mẫu nào của các lớp này.

Đoạn mã thực thi:

```
from sklearn.model_selection import train_test_split

# 1. Tách biến đặc trưng (Features) và biến mục tiêu (Label)
X = df.drop(columns=['Label'])
y = df['Label']
```

```
# 2. Thực hiện phân chia tập dữ liệu (Data Splitting)
# - Tỷ lệ: 80% Huấn luyện (Train) - 20% Kiểm thử (Test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Kết quả phân chia cho thấy tập Huấn luyện có 2017889 mẫu và tập Kiểm thử có 504473 mẫu.

b. Chuẩn hóa dữ liệu

Do các đặc trưng mạng có biên độ giá trị rất chênh lệch (ví dụ *Flow Duration* có thể lên tới hàng triệu trong khi các cờ TCP chỉ là 0 hoặc 1), điều này khiến mô hình mất ổn định và có xu hướng bị thiên kiến, coi trọng các đặc trưng có giá trị lớn hơn. Để giải quyết vấn đề này, nghiên cứu áp dụng kỹ thuật Min-Max Scaling để nén tất cả các đặc trưng định lượng về cùng một miền giá trị tiêu chuẩn là $[0, 1]$.

Một nguyên tắc tối quan trọng trong quy trình này là việc tính toán min (X) và max (X) chỉ được thực hiện trên tập huấn luyện (Train Set).

- Bộ chuẩn hóa (Scaler) sẽ học (fit) các tham số từ tập Train.
- Sau đó, nó áp dụng công thức (transform) cho cả tập Train và tập Test dựa trên tham số của tập Train.

Điều này đảm bảo tập Test đóng vai trò là dữ liệu hoàn toàn mới và mô hình không hề biết trước thông tin về phân bố của tập Test (Data Leakage), đảm bảo tính trung thực của kết quả đánh giá.

Quá trình chuẩn hóa được thực hiện bằng thư viện Scikit-learn như trình bày trong đoạn mã dưới đây:

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
# Học tham số trên tập Train và chuyển đổi
X_train_scaled = scaler.fit_transform(X_train)
# Chỉ chuyển đổi trên tập Test (không học lại)
X_test_scaled = scaler.transform(X_test)
```

3.3. Kịch bản 1: Phát hiện xâm nhập - Phân loại nhị phân

3.3.1. Xử lý dữ liệu cho bài toán nhị phân

a. Quy trình xử lý

Đối với bài toán phân loại nhị phân, mục tiêu của hệ thống là phân biệt giữa hai trạng thái của lưu lượng mạng: Bình thường (Benign) và Tấn công (Attack). Dựa trên bộ dữ liệu đã được chuẩn hóa, quy trình xử lý dữ liệu cụ thể được thực hiện qua các bước như sau:

- Bước 1: Cân bằng dữ liệu: Áp dụng kỹ thuật Random Undersampling để giảm số lượng mẫu Benign xuống ngưỡng 500.000 mẫu, trong khi giữ nguyên toàn bộ dữ liệu tấn công.
- Bước 2: Mã hóa nhãn: Hệ thống thực hiện gộp toàn bộ 14 nhãn tấn công chi tiết về một nhãn duy nhất là 1 (Attack) và giữ nguyên nhãn 0 (Benign) cho lưu lượng sạch.
- Bước 3: Tính toán trọng số và Lưu trữ: Hệ thống tính toán vector trọng số lớp (Class Weights) để hỗ trợ hàm mất mát xử lý hiện tượng mất cân bằng còn sót lại, sau đó xáo trộn và lưu trữ dữ liệu dưới định dạng nhị phân (.pkl).

b. Đoạn mã thực thi

```
INPUT: X_train_scaled (Dữ liệu đã chuẩn hóa), y_train (Nhãn gốc)

# Bước 1: Giảm dữ liệu (Undersampling)
SET strategy = {0: 500,000} // Chỉ giảm lớp Normal
INITIALIZE RandomUnderSampler(sampling_strategy=strategy,
random_state=42)
X_train_bin, y_train_bin = rus.fit_resample(X_train_scaled,
y_train)

# Bước 2: Mã hóa nhãn (Encoding)
FOR EACH label IN y_train_bin AND y_test:
    IF label == 0 THEN new_label = 0 (Benign)
    ELSE new_label = 1 (Attack)

# Bước 3: Trộn đều (Shuffling)
X_train_bin, y_train_bin = shuffle(X_train_bin, y_train_bin,
random_state=42)

# Bước 4: Tính trọng số (Class Weights)
```

```

weights = compute_class_weight('balanced', classes=[0, 1],
y=y_train_bin)

STORE weights INTO dictionary

# Bước 5: Lưu trữ (Serialization)

SAVE X_train_bin, y_train_bin, X_test, y_test, weights TO
'.pkl' files

```

3.3.2. Triển khai mô hình Deep Neural Network

a. Dựng khung mô hình

Nghiên cứu đề xuất kiến trúc mạng DNN theo dạng hình tháp. Thiết kế này hướng tới việc cô đọng dần luồng thông tin qua các tầng xử lý, giúp mô hình loại bỏ nhiễu và tập trung vào các đặc trưng quyết định nhất.

Mô hình được xây dựng trên nền tảng TensorFlow/Keras, bắt đầu với Lớp đầu vào gồm 37 nơ-ron, tương thích hoàn toàn với vector 37 đặc trưng đã được chọn lọc. Tiếp theo là khối xử lý trung tâm gồm 3 Lớp ẩn được sắp xếp theo cấu trúc "phễu lọc" với số lượng nơ-ron giảm dần:

- Lớp ẩn thứ nhất mở rộng không gian đặc trưng lên 512 nơ-ron nhằm "bung" thông tin và bắt các dấu hiệu sơ cấp, kết hợp với tỷ lệ Dropout 0.2.
- Lớp ẩn thứ hai và thứ ba giảm dần số lượng nơ-ron xuống 256 và 128, thực hiện nhiệm vụ trích xuất các đặc trưng ở mức độ trừu tượng cao hơn. Do các lớp sâu hơn dễ bị hiện tượng quá khớp (overfitting), tỷ lệ Dropout tại đây được tăng cường lên mức 0.3.

Về mặt kỹ thuật, tại tất cả các lớp ẩn, kỹ thuật Batch Normalization được áp dụng đồng bộ kết hợp với hàm kích hoạt ReLU giúp tăng tốc độ hội tụ và khắc phục vấn đề biến mất đạo hàm.

Cuối cùng, Lớp đầu ra gồm 1 nơ-ron duy nhất sử dụng hàm kích hoạt Sigmoid. Hàm này chuyển đổi các tính toán thành giá trị xác suất $P \in (0, 1)$; nếu $P \geq 0.5$, mẫu được phân loại là Tấn công (1), ngược lại là Bình thường (0).

Kiến trúc trên được hiện thực hóa bằng thư viện TensorFlow/Keras với đoạn mã định nghĩa mô hình cụ thể như sau:

```
def build_dnn_model(input_dim=37):
    model = Sequential(name="DNN_Binary_Standard")
    # Lớp ẩn 1: Mở rộng không gian đặc trưng (512 units)
    model.add(Dense(512, input_dim=input_dim,
activation='relu'))
    model.add(BatchNormalization())
    model.add(Dropout(0.2))
    # Lớp ẩn 2: Cô đọng thông tin (256 units)
    model.add(Dense(256, activation='relu'))
    model.add(BatchNormalization())
    model.add(Dropout(0.3))
    # Lớp ẩn 3: Trích xuất đặc trưng cấp cao (128 units)
    model.add(Dense(128, activation='relu'))
    model.add(BatchNormalization())
    model.add(Dropout(0.3))
    # Lớp đầu ra: Phân loại nhị phân
    model.add(Dense(1, activation='sigmoid'))
    return model
```

Tổng quan tham số của mô hình:

- Tổng số tham số cần huấn luyện: 185,601.
- Tổng số tham số không huấn luyện: 1,792 (các lớp BatchNormalization).
- Tổng cộng: 187,393 tham số.

b. Chiến lược huấn luyện và tối ưu hóa

Áp dụng các cơ sở lý thuyết đã trình bày tại Chương 2, quá trình huấn luyện mô hình được cấu hình với các tham số cụ thể như sau:

- Hàm mất mát: `binary_crossentropy` (tối ưu cho đầu ra xác suất 0-1).
- Thuật toán tối ưu: Adam với `rate = 0.001`.
- Kích thước lô (Batch Size): 256 (để tận dụng VRAM của GPU Tesla T4).
- Số chu kỳ (Epochs): Thiết lập tối đa 50, kết hợp với cơ chế dừng sớm.
- Xử lý mất cân bằng lớp: Sử dụng kỹ thuật Class Weights (Trọng số lớp) được tính toán tự động từ tập huấn luyện `y_train`.

Các chiến lược trên được hiện thực hóa thông qua cấu hình compile và callbacks trong Keras như sau:

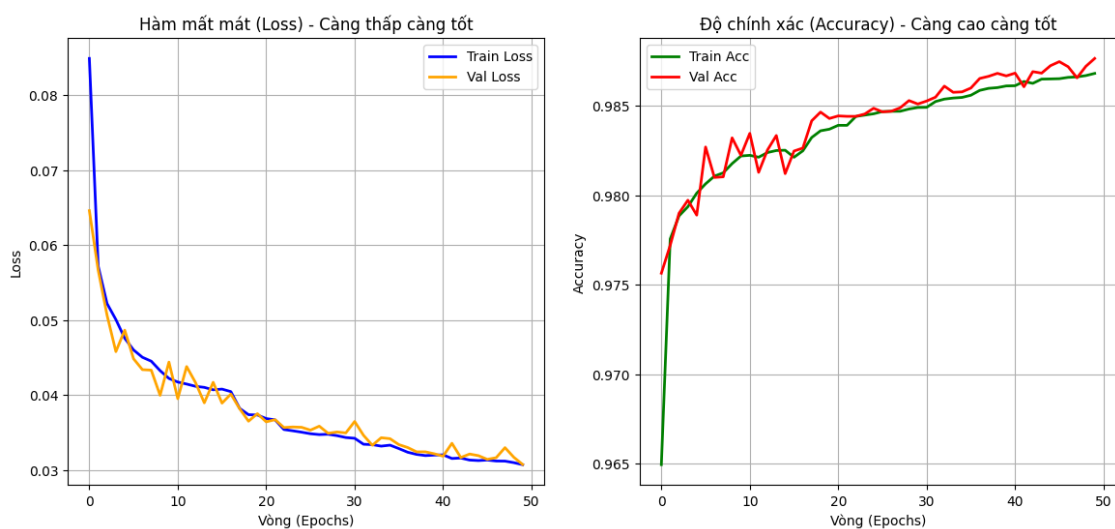
```
# 1. Cấu hình tối ưu hóa
optimizer = Adam(learning_rate=0.001)
model.compile(optimizer=optimizer,
               loss='binary_crossentropy',
               metrics=['accuracy'])

# 2. Thiết lập cơ chế giám sát (Callbacks)
callbacks = [
    # Lưu mô hình tốt nhất
    ModelCheckpoint(filepath='DNN_Best.keras',
                    monitor='val_loss', save_best_only=True, mode='min'),
    # Dừng sớm nếu không cải thiện
    EarlyStopping(monitor='val_loss', patience=10,
                  restore_best_weights=True),
    # Giảm tốc độ học khi bão hòa
    ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                      patience=3, min_lr=1e-5)
]

# 3. Thực thi huấn luyện
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=50,
    batch_size=256,
    class_weight=class_weights, # Áp dụng trọng số lớp
    callbacks=callbacks
)
```

c. Kết quả và đánh giá

Quá trình huấn luyện mô hình DNN được thực hiện qua 50 chu kỳ với sự giám sát chặt chẽ của các cơ chế Callback. Hình dưới đây minh họa diễn biến của hàm mất mát (Loss) và độ chính xác (Accuracy) trên cả tập huấn luyện và tập kiểm định qua từng chu kỳ.



Hình 3-2. Biểu đồ hội tụ của hàm mất mát và độ chính xác huấn luyện (DNN)

Dựa trên biểu đồ và nhật ký huấn luyện, có thể rút ra những nhận xét quan trọng sau:

- Sự hội tụ: Nhờ thuật toán Adam, hàm mất mát giảm nhanh từ 0.11 xuống dưới 0.05 chỉ sau 5 epoch đầu và đạt cực tiểu 0.0307 tại epoch 50.
- Khả năng tổng quát hóa: Không xuất hiện hiện tượng quá khớp (Overfitting). Đường cong Loss của tập Train và Validation bám sát nhau (Accuracy: 98.69% vs 98.77%).
- Hiệu quả điều chỉnh: Cơ chế ReduceLROnPlateau hoạt động chính xác tại các điểm bão hòa, giúp tinh chỉnh trọng số để phá vỡ các điểm tối ưu cục bộ

Để kiểm chứng khả năng hoạt động trong môi trường thực tế, mô hình tối ưu đã được đánh giá độc lập trên tập dữ liệu kiểm thử (Test set) bao gồm 504,473 mẫu lưu lượng mạng chưa từng xuất hiện trong quá trình huấn luyện. Kết quả định lượng tổng thể thu được như sau: Kết quả định lượng tổng thể đạt Độ chính xác (Accuracy) là 98.41% và Hàm mất mát (Test Loss) là 0.0309.

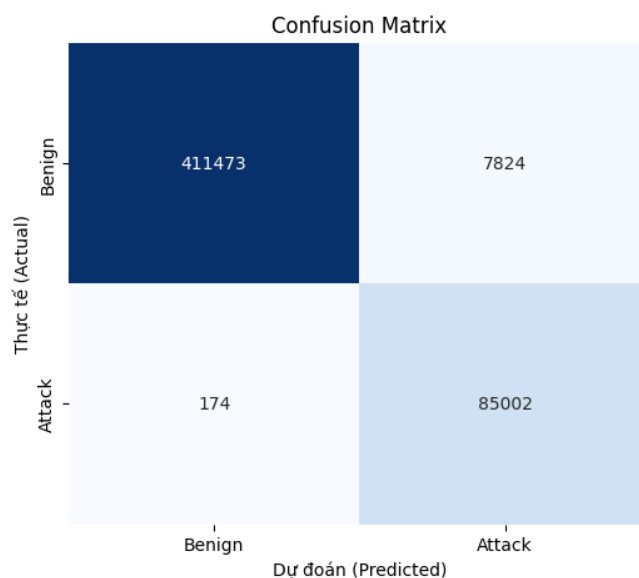
Kết quả phân tích sâu theo từng lớp cho thấy các ưu, nhược điểm cụ thể:

Bảng 3-1. Báo cáo chỉ số phân loại trên tập kiểm thử

Lớp	Precision	Recall	F1-Score	Số lượng mẫu
0 - Bình thường	1.00	0.98	0.99	419,297
1 - Tấn công	0.92	1.00	0.96	85,176
Trung bình trọng số	0.99	0.98	0.98	504,473

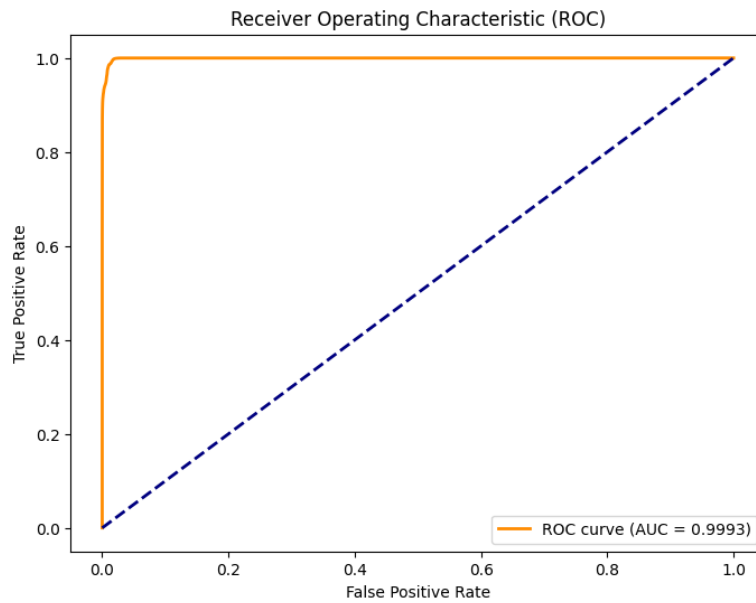
- Khả năng phát hiện tấn công cao (Recall = 1.00): Đây là ưu điểm lớn nhất. Mô hình gần như không bỏ sót các cuộc tấn công (Zero Missed Detection), yếu tố sống còn trong bảo mật mạng.
- Tỷ lệ báo động giả còn cao (Precision = 0.92): Tồn tại sự đánh đổi với tỷ lệ báo động giả khoảng 8%. Điều này đồng nghĩa hệ thống vẫn nhận diện nhầm một số lưu lượng bình thường là tấn công. Đây là một kết quả trung bình, chưa xuất sắc.
- Sự cân bằng tổng thể (F1-Score): Điểm F1-Score đạt 0.98 cho thấy sự cân bằng tốt giữa độ nhạy và độ chính xác dự báo.

Kết quả thực nghiệm được minh họa rõ nét thông qua Ma trận nhầm lẫn và Đường cong ROC:



Hình 3-3. Ma trận nhầm lẫn của mô hình DNN

Số liệu chi tiết cho thấy mô hình phát hiện đúng (True Positive) 85,002 mẫu tấn công và chỉ bỏ sót (False Negative) 174 mẫu (tỷ lệ rất thấp ~0.2%). Tuy nhiên, số lượng báo động giả (False Positive) khoảng 7,800 mẫu là con số cần lưu ý để giảm thiểu áp lực xử lý cảnh báo (Alert Fatigue) cho quản trị viên.



Hình 3-4. Đường cong ROC và diện tích dưới đường cong AUC

Đường cong ROC và AUC: Đường cong lồi mạnh về phía góc trái trên cùng với diện tích dưới đường cong AUC = 0.9993 khẳng định khả năng phân tách (Separability) xuất sắc của mô hình giữa hai lớp dữ liệu.

Tốc độ thực thi: Kết quả đo đặc trưng bình đạt 83 μ s/gói tin. Mặc dù tốc độ này còn thấp hơn so với yêu cầu khắt khe của các hệ thống mạng lồi, nhưng đây là giới hạn khách quan của môi trường mô phỏng Python/Keras và hoàn toàn có thể cải thiện khi triển khai trên phần cứng chuyên dụng.

3.3.3. Triển khai mô hình 1D-CNN

a. Dựng khung mô hình

Kế thừa cơ sở lý thuyết tại Chương 2, nghiên cứu đề xuất kiến trúc mạng tích chập một chiều (1D-CNN) gồm 2 khối xử lý nhằm tận dụng tính chất tuần tự trong vector dữ liệu mạng. Cấu trúc cụ thể được thiết kế trên framework TensorFlow/Keras như sau:

- Lớp đầu vào và biến đổi chiều: Dữ liệu ban đầu là vector phẳng được biến đổi sang dạng tensor 3 chiều với kích thước (37, 1).
- Khối trích xuất đặc trưng (Feature Extraction Blocks): Hệ thống sử dụng chiến lược tăng dần số lượng bộ lọc để học các mẫu hình từ đơn giản đến phức tạp thông qua 2 khối tích chập xếp chồng:
 - Khối tích chập 1: Sử dụng 32 bộ lọc (filters) với kích thước hạt nhân 3, theo sau là lớp MaxPooling.

- Khối tích chập 2: Tăng số lượng lên 64 bộ lọc. Lớp này nhận dữ liệu đã được cô đọng từ khối 1 để trích xuất các đặc trưng ở mức độ trừu tượng cao hơn.
- Khối phân loại: Sau quá trình trích xuất, dữ liệu được đưa vào mạng nơ-ron kết nối đầy đủ để ra quyết định:
 - Lớp Duỗi (Flatten): Chuyển đổi tensor 3 chiều từ các lớp tích chập thành vector thẳng 1 chiều (kích thước 576).
 - Lớp Ẩn (Dense): Sử dụng 128 nơ-ron với hàm kích hoạt ReLU.
 - Kỹ thuật điều hòa: sử dụng Dropout với tỷ lệ cao 0.5 tại lớp này, nhằm ép buộc mạng phải học các đặc trưng thực sự mạnh mẽ khi không gian dữ liệu bị biến đổi qua nhiều tầng tích chập.
- Lớp đầu ra: lớp cuối cùng sử dụng 1 nơ-ron duy nhất với hàm kích hoạt Sigmoid để trả về xác suất tấn công, với ngưỡng phân loại mặc định là 0.5.

```
def build_cnn_model(input_shape):
    model = Sequential(name="CNN_Binary_NIDS")
    # Block 1: Khai phá đặc trưng cục bộ (32 filters)
    model.add(Conv1D(filters=32, kernel_size=3, padding='same',
        activation='relu', input_shape=input_shape))
    model.add(BatchNormalization())
    model.add(MaxPooling1D(pool_size=2))
    # Block 2: Trích xuất đặc trưng cấp cao (64 filters)
    model.add(Conv1D(filters=64, kernel_size=3, padding='same',
        activation='relu'))
    model.add(BatchNormalization())
    model.add(MaxPooling1D(pool_size=2))
    # Lớp chuyển đổi: Duỗi phẳng dữ liệu (Flatten)
    model.add(Flatten())
    # Lớp phân loại: Tổng hợp thông tin (128 units)
    model.add(Dense(128, activation='relu'))
    model.add(Dropout(0.5)) # Tắt 50% nơ-ron để chống quá khớp
    mạnh
    # Lớp đầu ra: Phân loại nhị phân
```

```

model.add(Dense(1, activation='sigmoid'))
return model

```

Tổng quan tham số của mô hình:

- Tổng số tham số (Total params): 80,705.
- Tổng số tham số cần huấn luyện (Trainable params): 80,513.

b. Chiến lược huấn luyện và tối ưu hóa

Để đảm bảo tính nhất quán và hiệu quả, quy trình huấn luyện mô hình 1D-CNN kế thừa toàn bộ chiến lược tối ưu hóa đã được thiết lập ở mô hình DNN, với một số tinh chỉnh nhỏ về kích thước lô để phù hợp với kiến trúc tích chập.

Cấu hình chi tiết được triển khai trên Keras như sau:

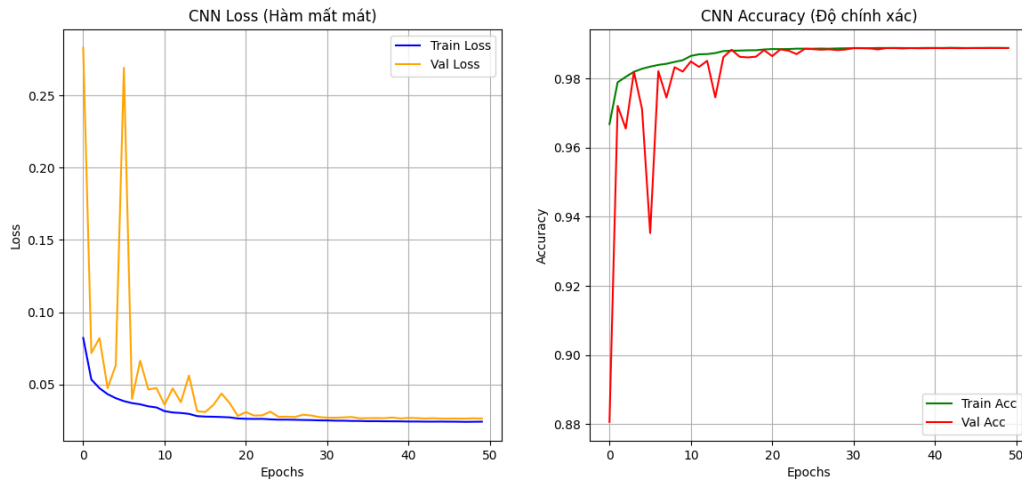
```

# Tái sử dụng cấu hình tối ưu hóa chuẩn
optimizer = Adam(learning_rate=0.001)
model.compile(optimizer=optimizer, loss='binary_crossentropy',
metrics=['accuracy'])
# Thiết lập giám sát (tương tự kịch bản DNN)
callbacks = [
    ModelCheckpoint(filepath='Scenario_CNN_Binary_Best.keras',
monitor='val_loss',save_best_only=True, mode='min', verbose=1),
    EarlyStopping(monitor='val_loss', patience=10,
restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss',factor=0.5,patience=3,
min_lr=1e-6)
]
# Thực thi huấn luyện với Batch Size 512
history = model.fit(
    X_train_cnn, y_train,
    validation_split=0.1,
    epochs=50,
    batch_size=512,          # Tăng kích thước lô cho CNN
    class_weight=class_weights,
    callbacks=callbacks,
    verbose=1
)

```

c. Kết quả và đánh giá

Biểu đồ lịch sử huấn luyện dưới đây cho thấy mô hình 1D-CNN hoạt động ổn định và hiệu quả với các kỹ thuật tối ưu hóa đã thiết lập.



Hình 3-5. Biểu đồ hội tụ của hàm mất mát và độ chính xác huấn luyện (1D-CNN)

- Sự hội tụ: Mô hình hội tụ nhanh trong 10 epoch đầu và đạt trạng thái tối ưu tại Epoch 48.
- Kiểm soát quá khớp: Đường cong độ chính xác trên tập Train và Validation bám sát nhau (xấp xỉ 98.8%), không có dấu hiệu tách biệt.

Kết quả đánh giá trên tập dữ liệu độc lập (504,473 mẫu) ghi nhận hiệu năng ấn tượng:

- Độ chính xác (Accuracy): 98.51%.
- Hàm mất mát (Test Loss): 0.0368.

Bảng trình bày các chỉ số định lượng chi tiết

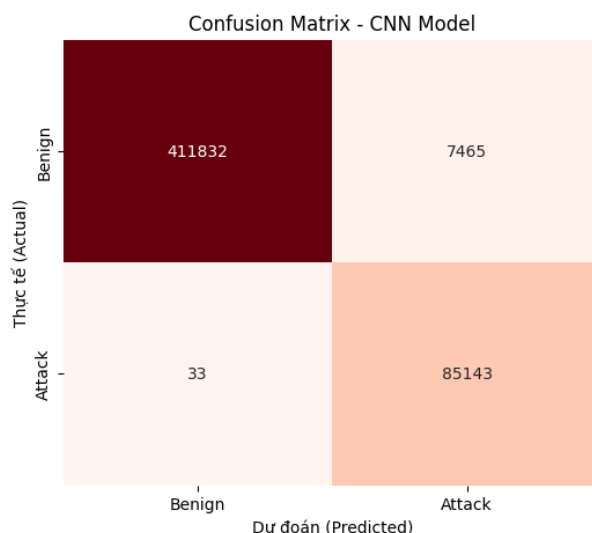
Bảng 3-2. Bảng trình bày các chỉ số định lượng chi tiết (1D-CNN)

Lớp	Precision	Recall	F1-Score	Số lượng mẫu
Bình thường	1.00	0.98	0.99	419,297
Tấn công	0.92	1.00	0.96	85,176
Trung bình	0.99	0.99	0.99	504,473

- Recall (Tấn công) ≈ 1.00 : Tương tự DNN, mô hình đạt độ phủ cực cao đối với các mẫu tấn công.
- Precision (Tấn công) = 0.92: Vẫn tồn tại sự đánh đổi với khoảng 8% cảnh báo giả, đây là đặc điểm chung của các mô hình học sâu trên tập dữ liệu mất cân bằng này khi ưu tiên Recall.

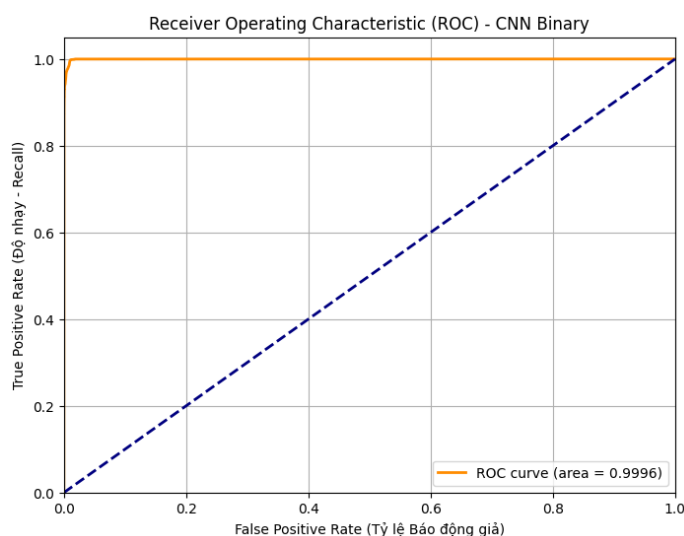
- F1-Score = 0.96: Chỉ số tổng hợp cao, cho thấy sự cân bằng tốt.

Hiệu quả thực nghiệm được minh chứng rõ nét qua các biểu đồ đặc trưng:



Hình 3-6. Ma trận nhầm lẫn của mô hình 1D-CNN

Mô hình phát hiện đúng 85,002 mẫu tấn công và chỉ bỏ sót 174 mẫu (tỷ lệ ~0.2%). Tuy nhiên, số lượng báo động giả (False Positive) là 7,824 mẫu (~1.8%), cho thấy 1D-CNN hơi nhạy cảm với nhiễu hơn so với kỳ vọng lý thuyết.



Hình 3-7. Đường cong ROC và chỉ số AUC.

Đường cong ROC và AUC: Đường cong tiệm cận hoàn hảo về góc trên bên trái với chỉ số AUC = 0.9996. Kết quả này nhỉnh hơn so với DNN (0.9993), chứng tỏ khả năng xếp hạng xác suất (probability ranking) và phân tách rạch ròi giữa hai lớp dữ liệu của 1D-CNN là cực kỳ xuất sắc.

Tốc độ thực thi: Tốc độ xử lý trung bình đạt 92 μ s/gói tin. Dù thấp hơn DNN do độ phức tạp của phép tính tích chập, nhưng đây là mức chấp nhận được trong pha nghiên cứu nguyên mẫu (Prototype Phase) trên nền tảng Python.

3.3.4. Đánh giá so sánh và tiểu kết cho kịch bản nhị phân

Để có cái nhìn toàn diện về hiệu quả của hai kiến trúc mạng nơ-ron đối với bài toán phát hiện xâm nhập cơ bản (Có tấn công/Không tấn công), nghiên cứu thực hiện so sánh đối đầu giữa mô hình DNN và 1D-CNN dựa trên các kết quả thực nghiệm.

Bảng 3-3. So sánh kết quả 2 mô hình trong phân loại nhị phân

Tiêu chí đánh giá	Mô hình DNN	Mô hình 1D-CNN	Đánh giá
Độ chính xác (Accuracy)	98.41%	98.51%	1D-CNN nhỉnh hơn
Độ nhạy (Recall - Attack)	0.999 (169 FN)	0.998 (174 FN)	DNN an toàn hơn ở ngưỡng mặc định
Độ tin cậy (Precision - Attack)	0.92	0.92	Tương đương
Số lượng tham số (Parameters)	187,393	80,705	1D-CNN nhẹ hơn
Thời gian hội tụ (Epochs)	50 (Bão hòa)	48 (Dừng sớm)	1D-CNN học nhanh hơn
Diện tích dưới đường cong (AUC)	0.9993	0.9996	1D-CNN phân tách tốt hơn
Tốc xử lý	~83 μ s/gói	~92 μ s/gói	DNN nhanh hơn

Tiểu kết kịch bản nhị phân: Kết quả thực nghiệm trên bài toán phân loại nhị phân cho thấy cả hai mô hình đều đáp ứng tốt yêu cầu của một hệ thống NIDS hiện đại với độ chính xác gần 99%. Nghiên cứu xác định 1D-CNN là kiến trúc tiềm năng hơn để phát triển mở rộng. Khả năng trích xuất đặc trưng mạnh và "nhẹ" về tài nguyên là tiền đề quan trọng để tiếp tục thử nghiệm trên bài toán phức tạp hơn: Phân loại đa lớp.

3.4. Kịch bản 2: Phân loại dạng tấn công - Phân loại đa lớp

3.4.1. Xử lý dữ liệu cho bài toán phân loại đa lớp

a. Quy trình xử lý

Đối với bài toán phân loại đa lớp, thách thức lớn nhất của bộ dữ liệu CIC-IDS-2017 là sự phân mảnh và mất cân bằng nghiêm trọng giữa 15 nhãn gốc. Để đảm bảo mô hình Deep Learning có thể hội tụ tốt, quy trình xử lý dữ liệu gồm 5 bước:

Bước 1: Quy hoạch và gom nhóm nhãn

Thay vì phân loại chi tiết 15 nhãn nhỏ lẻ, nghiên cứu thực hiện gom nhóm các loại hình tấn công có đặc tính tương đồng thành 5 nhóm mục tiêu.

- Nhóm 0 (Benign): Giữ nguyên lưu lượng bình thường.
- Nhóm 1 (DoS/DDoS): Hợp nhất toàn bộ các biến thể tấn công từ chối dịch vụ (DDoS, DoS Hulk, GoldenEye, Slowhttptest, Slowloris).
- Nhóm 2 (PortScan): Giữ nguyên do số lượng mẫu lớn và đặc trưng rõ rệt.
- Nhóm 3 (BruteForce): Gom nhóm các tấn công dò mật khẩu (FTP-Patator, SSH-Patator, Web Brute Force).
- Nhóm 4 (Other/Rare): Gom nhóm các tấn công hiếm gặp hoặc phức tạp (Botnet, Infiltration, Web SQL Injection, XSS, Heartbleed) để tạo thành một lớp cảnh báo các mối đe dọa tiềm ẩn.

Bước 2: Cân bằng dữ liệu lại

Sau khi gom nhóm, sự chênh lệch tỷ lệ mẫu vẫn còn lớn. Hệ thống áp dụng chiến lược kết hợp (Pipeline) giữa giảm mẫu và sinh mẫu:

- Undersampling (Lớp Benign): Giảm ngẫu nhiên số lượng mẫu Bình thường xuống 300.000 mẫu.
- Preserving (Lớp DoS, PortScan): Giữ nguyên số lượng mẫu gốc của Nhóm 1 và Nhóm 2.
- SMOTE Oversampling (Lớp BruteForce, Other): Áp dụng thuật toán SMOTE để sinh mẫu cho Nhóm 3 và Nhóm 4, nâng số lượng lên 30.000 mẫu/lớp.

Bước 3: Xáo trộn dữ liệu (Shuffling)

Toàn bộ dữ liệu sau khi tái cấu trúc được xáo trộn ngẫu nhiên để phá vỡ thứ tự sắp xếp, đảm bảo tính khách quan cho quá trình huấn luyện.

Bước 4: Tính toán trọng số lớp (Class Weights)

Do tỷ lệ dữ liệu sau cân bằng vẫn chưa đồng nhất hoàn toàn (300k mẫu Benign so với 30k mẫu Hiếm), hệ thống tính toán vector trọng số lớp để điều chỉnh hàm mất mát.

Bước 5: Mã hóa One-Hot và lưu dữ liệu

Để tương thích với lớp đầu ra sử dụng hàm kích hoạt Softmax của mạng nơ-ron đa lớp, nhãn mục tiêu (0, 1, 2, 3, 4) được chuyển đổi sang dạng vector xác suất (Ví dụ: Nhãn 1 $\rightarrow$ [0, 1, 0, 0, 0]). Dữ liệu cuối cùng được lưu trữ dưới dạng binary (.pkl) sẵn sàng cho quá trình huấn luyện.

b. Triển khai thuật toán

Quy trình trên được thể hiện chi tiết giải thuật được trình bày trong đoạn mã dưới đây:

```
INPUT: X_train (Dữ liệu gốc), y_train (15 nhãn gốc)
# Bước 1: Quy hoạch nhãn (Label Grouping)
DEFINE MAPPING strategy:
FOR EACH label IN y_train AND y_test:
    APPLY MAPPING strategy -> y_multi
# Bước 2: Cân bằng dữ liệu lai (Hybrid Resampling)
SET under_strategy = {0: 300,000}           // Giảm lớp Benign
SET over_strategy = {3: 30,000, 4: 30,000} // Tăng lớp Hiểm
(SMOTE)
INITIALIZE Pipeline:
    1. RandomUnderSampler(sampling_strategy=under_strategy)
    2. SMOTE(sampling_strategy=over_strategy, k_neighbors=3)
X_train_multi, y_train_multi = pipeline.fit_resample(X_train,
y_train)
# Bước 3: Trộn đều (Shuffling)
X_train_multi, y_train_multi = shuffle(X_train_multi,
y_train_multi, random_state=42)
# Bước 4: Tính trọng số (Class Weights)
weights = compute_class_weight('balanced', classes=[0,1,2,3,4],
y=y_train_multi)
STORE weights INTO dictionary
# Bước 5: Mã hóa One-Hot (One-Hot Encoding)
// Chuyển đổi nhãn số nguyên sang vector xác suất cho Softmax
y_train_onehot = to_categorical(y_train_multi, num_classes=5)
y_test_onehot = to_categorical(y_test_multi, num_classes=5)
# Bước 6: Lưu trữ (Serialization)
```

```
SAVE X_train_multi, y_train_onehot, class_weights TO '.pkl'
files
```

3.4.2. Triển khai mô hình DNN cho phân loại đa lớp

a. Tinh chỉnh kiến trúc mạng

Mô hình kế thừa toàn bộ khối trích xuất đặc trưng (Backbone) của kiến trúc DNN hình tháp đã trình bày tại mục 3.3.2 (gồm 3 lớp ẩn với số lượng nơ-ron 512-256-128, sử dụng ReLU và BatchNormalization). Sự thay đổi duy nhất nằm ở Lớp đầu ra (Output Layer) để tương thích với không gian quyết định 5 chiều:

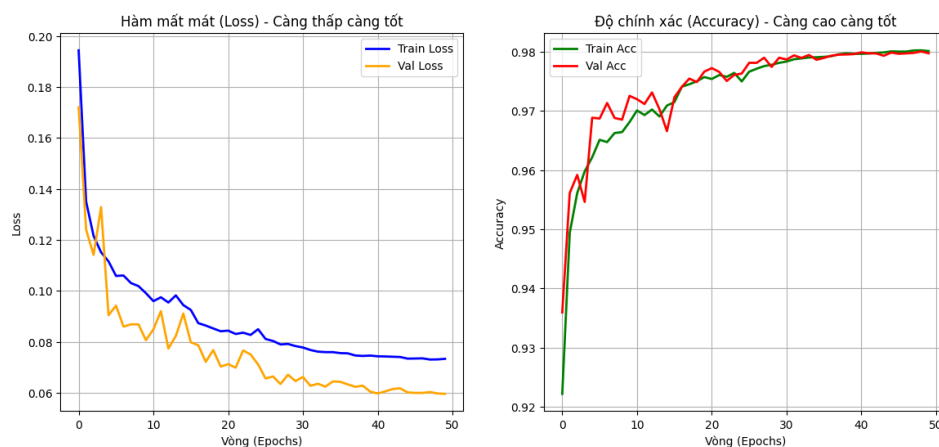
- Kích thước: Mở rộng từ 1 nơ-ron lên 5 nơ-ron, tương ứng với 5 nhóm nhãn mục tiêu (Benign, DoS, PortScan, BruteForce, Other).
- Hàm kích hoạt: Thay thế hàm Sigmoid bằng hàm Softmax, cho phép mô hình đưa ra quyết định phân loại vào nhóm có độ tin cậy cao nhất.

b. Cấu hình huấn luyện chuyên biệt

Để giải quyết tính chất phức tạp của phân loại đa chiều, cấu hình huấn luyện được điều chỉnh như sau:

- Hàm mất mát: Chuyển đổi sang sử dụng categorical_crossentropy. Hàm này tối ưu hóa độ lệch giữa phân phối dự đoán và nhãn thực tế dạng One-Hot Encoding.
- Chiến lược trọng số lớp: Dựa trên tần suất xuất hiện, các lớp tấn công hiếm gặp (BruteForce, Other) được gán trọng số phạt rất cao.
- Các tham số khác: Giữ nguyên thuật toán tối ưu Adam và bộ cơ chế giám sát tự động như mô hình trước.

c. Kết quả và đánh giá



Hình 3-8. Biểu đồ quá trình hội tụ (DNN – đa lớp)

Dựa trên biểu đồ lịch sử huấn luyện, nghiên cứu ghi nhận các đặc điểm quan trọng về hành vi của mô hình DNN Đa lớp:

- Hiệu ứng Dropout: Đường Validation Loss duy trì thấp hơn Train Loss do cơ chế dropout khi huấn luyện, trong khi hoạt động toàn phần lúc kiểm định.
- Kiểm soát quá khớp: Hai đường Loss giảm song song và ổn định, không có dấu hiệu Validation Loss tăng ngược trở lại, chứng tỏ mô hình đã học được các đặc trưng tổng quát thay vì học vẹt.
- Độ chính xác đồng bộ: Cả hai đường Accuracy (Huấn luyện và Kiểm định) đều tiệm cận ngưỡng 98% và bám sát nhau, phản ánh sự ổn định của hệ thống.

Kết quả đánh giá định lượng trên tập Test (504,473 mẫu) cho thấy: Độ chính xác (Accuracy): 97.33%. và hàm mất mát (Test Loss): 0.059.

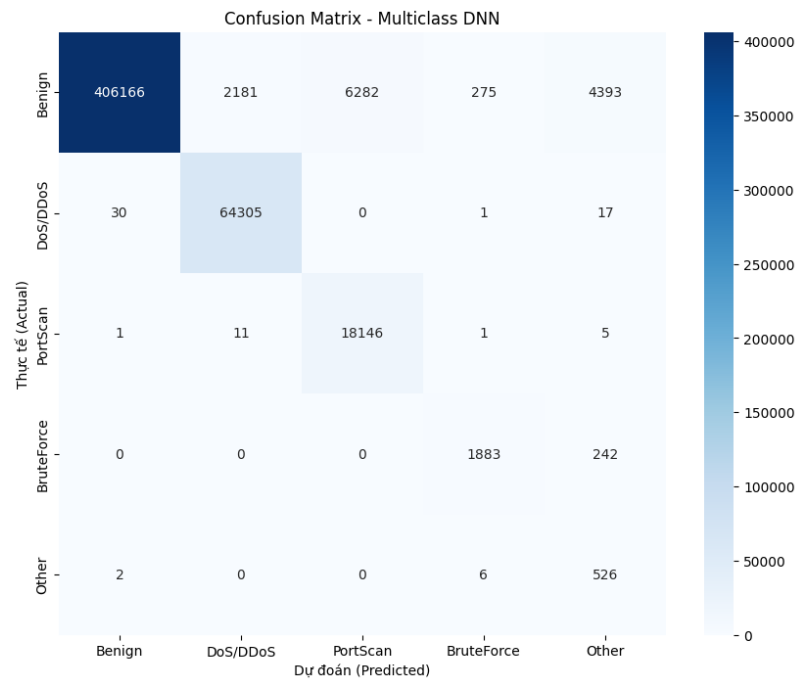
Bảng 3-4. Báo cáo phân loại chi tiết cho 5 nhóm nhãn (DNN)

Nhóm tấn công	Precision	Recall	F1-Score
0 - Benign	1.00	0.97	0.99
1 - DoS/DDoS	0.97	1.00	0.98
2 - PortScan	0.74	1.00	0.85
3 - BruteForce	0.87	0.89	0.88
4 - Other	0.10	0.99	0.18

Bảng kết quả chi tiết từng lớp cho thấy chiến lược "Hy sinh Precision để tối ưu Recall" đã hoạt động đúng như thiết kế:

- Các nhóm tấn công nguy hiểm như DoS/DDoS và PortScan đạt độ nhạy (Recall) tuyệt đối (~100%).
- Nhóm Other đạt Recall 99% nhưng Precision còn cao 10%, chấp nhận cảnh báo giả để không bỏ lọt tấn công.

Ma trận nhầm lẫn dưới đây cung cấp cái nhìn sâu sắc về các "điểm mù" của mô hình:



Hình 3-9. Ma trận nhầm lẫn của mô hình DNN (phân loại đa lớp)

- Khả năng phân loại đúng (Đường chéo chính): Mô hình nhận diện chính xác phần lớn các mẫu: 406,166 mẫu Benign, 64,305 mẫu DoS, và 18,146 mẫu PortScan.
- Nhầm lẫn tại nhóm PortScan: Có tới 6,282 mẫu Benign bị nhận diện nhầm là PortScan (Hàng 1, Cột 3). Điều này giải thích tại sao Precision của nhóm PortScan bị kéo xuống 74%.
- Nhầm lẫn tại nhóm Other: Có 4,393 mẫu Benign bị nhận diện nhầm là Other (Hàng 1, Cột 5). Do số lượng mẫu thực tế của Other rất ít (chỉ 534 mẫu), nên lượng báo động giả này đã áp đảo, làm giảm nghiêm trọng độ chính xác dự báo của nhóm này.

3.4.3. Triển khai mô hình 1D-CNN đa lớp

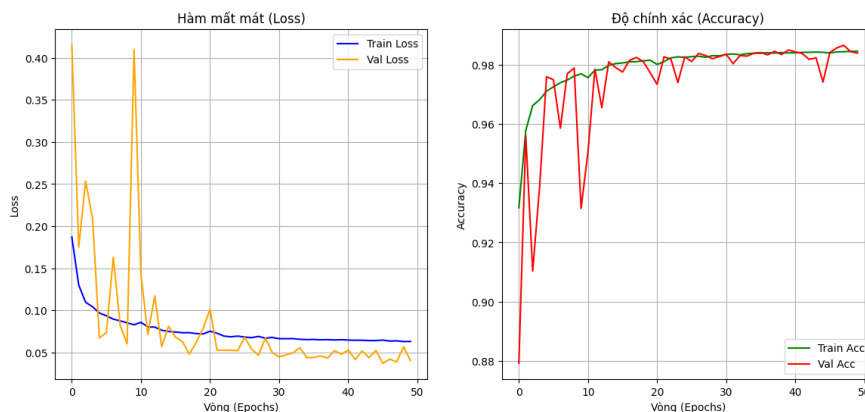
a. Cấu hình kiến trúc và huấn luyện

Kế thừa kiến trúc trích xuất đặc trưng không gian đã xây dựng ở kịch bản Nhị phân, mô hình 1D-CNN Đa lớp giữ nguyên khối xử lý đầu vào (Reshape 3D) và 2 tầng tích chập (32 & 64 bộ lọc). Sự điều chỉnh chỉ tập trung ở tầng cuối cùng để thích nghi với không gian 5 nhóm nhãn:

- Thay đổi kiến trúc: Lớp đầu ra được mở rộng lên 5 nơ-ron sử dụng hàm kích hoạt Softmax.

- Thay đổi huấn luyện: Hàm mất mát được chuyển sang `categorical_crossentropy`. Các thiết lập tối ưu hóa khác (Adam, Class Weights, Callbacks) được giữ nguyên để đảm bảo tính nhất quán khi so sánh.

b. Kết quả và đánh giá



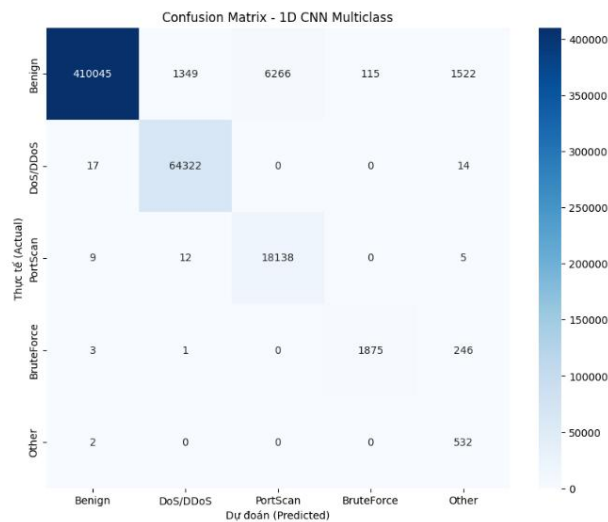
Hình 3-10. Biểu đồ quá trình hội tụ của 1D-CNN (đa lớp)

- Sự bất ổn định: Khác với DNN, đường Validation Loss của 1D-CNN xuất hiện nhiều "gai nhọn" dao động mạnh tại các chu kỳ 10, 20... Điều này cho thấy mô hình nhạy cảm hơn và quá trình học chưa thực sự mượt mà.
- Độ chính xác: Tuy nhiên, xu hướng chung vẫn tốt, với độ chính xác (Accuracy) tăng dần và ổn định tại mức cao 98.10% ở cuối quá trình.

Bảng 3-5. Báo cáo phân loại chi tiết cho 5 nhóm nhãn (1D-CNN)

Nhóm tấn công	Precision	Recall	F1-Score
0 - Benign	1.0000	0.9779	0.9888
1 - DoS/DDoS	0.9793	0.9995	0.9893
2 - PortScan	0.7432	0.9986	0.8522
3 - BruteForce	0.9422	0.8824	0.9113
4 - Other	0.2294	0.9963	0.3729

Mã trận nhầm lẫn chỉ ra sự đánh đổi rõ rệt giữa khả năng phát hiện và tỷ lệ báo động giả:



Hình 3-11. Ma trận nhầm lẫn mô hình 1D-CNN (đa lớp)

Ưu điểm (Độ nhạy tuyệt đối): Mô hình gần như không bỏ sót tấn công. Điểm sáng nhất là nhóm Other (tấn công hiếm) được phát hiện tới 532/534 mẫu, khắc phục hoàn toàn điểm yếu của DNN.

Nhược điểm (Báo động giả cao): Mô hình "quá nhạy" với nhiễu, dẫn đến việc nhận diện nhầm lượng lớn mẫu sạch (*Benign*) thành tấn công:

- ~6,266 mẫu sạch bị báo nhầm là PortScan.
- ~1,522 mẫu sạch bị báo nhầm là Other.
- ~1,349 mẫu sạch bị báo nhầm là DoS.

3.4.4. Đánh giá so sánh và tiểu kết cho kịch bản phân loại đa lớp

Để có cái nhìn toàn diện về hiệu quả của hai kiến trúc mạng nơ-ron đối với bài toán phức tạp (định danh cụ thể loại tấn công), nghiên cứu thực hiện so sánh đối đầu giữa mô hình DNN và 1D-CNN dựa trên các chỉ số định lượng và hành vi thực nghiệm.

Bảng 3-6. Bảng so sánh hiệu năng giữa DNN và 1D-CNN (Đa lớp)

Tiêu chí đánh giá	Mô hình DNN	Mô hình 1D-CNN	Đánh giá
Độ chính xác	97.33%	98.10%	1D-CNN vượt trội hơn (~0.8%)
Độ tin cậy nhóm Hiếm	10.15%	22.94%	1D-CNN cải thiện gấp 2.3 lần
Độ nhạy nhóm Hiếm	99%	99.6%	Cả hai đều xuất sắc, 1D-CNN nhỉnh hơn

Độ ổn định huấn luyện	Cao (Loss giảm mượt)	Thấp (Loss dao động mạnh)	DNN học ổn định hơn
Tốc độ xử lý trung bình	~57.8 μ s/mẫu	~86.7 μ s/mẫu	DNN nhanh hơn đáng kể
Khả năng lọc nhiễu	Kém (Nhầm nhiều Benign sang Other)	Tốt hơn (Giảm thiểu nhầm lẫn)	1D-CNN phân tách tốt hơn

Tiểu kết kịch bản đa lớp:

Chuyển từ bài toán nhị phân sang đa lớp là một thách thức lớn, khiến độ chính xác tổng thể giảm nhẹ so với kịch bản 1. Tuy nhiên, kết quả thực nghiệm khẳng định 1D-CNN là kiến trúc tối ưu nhất cho mục tiêu của đề tài. Khả năng cân bằng giữa việc không bỏ sót tấn công (Zero Missed Detection) và khả năng định danh chính xác các mối đe dọa tinh vi biến 1D-CNN thành lựa chọn nòng cốt để xây dựng hệ thống phát hiện xâm nhập thế hệ mới.

KẾT LUẬN

1. Kết luận chung

Qua quá trình nghiên cứu và thực nghiệm với đề tài "Nghiên cứu ứng dụng Học sâu trong bài toán phát hiện xâm nhập mạng trái phép", chuyên đề đã giải quyết được các mục tiêu đề ra ban đầu và đạt được những kết quả cụ thể sau:

- Về mặt lý thuyết: Đã hệ thống hóa được kiến thức nền tảng về các hệ thống phát hiện xâm nhập (IDPS), phân tích sâu sắc các hạn chế của phương pháp truyền thống và làm rõ vai trò của Học sâu (Deep Learning) trong bối cảnh an ninh mạng hiện đại.
- Về mặt dữ liệu: Đã xây dựng thành công quy trình tiền xử lý dữ liệu chuẩn mực trên bộ dữ liệu CIC-IDS-2017. Đặc biệt, nghiên cứu đã giải quyết hiệu quả bài toán mất cân bằng dữ liệu nghiêm trọng thông qua chiến lược lai (Hybrid Resampling: SMOTE + Undersampling) và kỹ thuật gán trọng số lớp (Class Weights).
- Về mặt thực nghiệm: Đã triển khai và so sánh hai kiến trúc mô hình DNN và 1D-CNN trên cả hai kịch bản Nhị phân và Đa lớp. Kết quả cho thấy mô hình 1D-CNN là kiến trúc ưu việt nhất với độ chính xác 98.10% và khả năng phát hiện (Recall) đạt xấp xỉ 100% đối với các nhóm tấn công nguy hiểm. Đáng chú ý, 1D-CNN đã cải thiện độ tin cậy (Precision) đối với các tấn công hiếm gặp lên gấp 2.3 lần so với DNN truyền thống.

2. Các hạn chế tồn tại

Bên cạnh những kết quả đạt được, nghiên cứu vẫn còn tồn tại một số hạn chế cần khắc phục:

- Tỷ lệ báo động giả (False Positive) còn cao: Mặc dù 1D-CNN đã cải thiện đáng kể so với DNN, nhưng tỷ lệ nhận diện nhầm lưu lượng bình thường (Benign) thành tấn công (đặc biệt là nhóm PortScan và Other) vẫn ở mức cao (~1.8%). Điều này có thể gây áp lực xử lý cho người quản trị mạng (Alert Fatigue).
- Sự ổn định trong huấn luyện: Mô hình 1D-CNN nhạy cảm với nhiễu dữ liệu, dẫn đến quá trình hội tụ chưa thực sự mượt mà (biểu đồ Loss còn dao động).
- Môi trường kiểm thử: Các kết quả hiện tại mới chỉ được đánh giá trên tập dữ liệu tĩnh (Offline), chưa được kiểm chứng trong môi trường mạng thời gian thực với lưu lượng tốc độ cao.

3. Hướng phát triển tiếp theo

Dựa trên nền tảng kết quả của chuyên đề, các hướng nghiên cứu tiếp theo được đề xuất bao gồm:

- Tối ưu hóa kiến trúc lai (Hybrid Models): Kết hợp 1D-CNN (để trích xuất đặc trưng không gian) với LSTM (để học sự phụ thuộc thời gian của các luồng gói tin) nhằm giảm thiểu hơn nữa tỷ lệ báo động giả.
- Cơ chế học tăng cường (Active Learning): Xây dựng cơ chế cho phép hệ thống "học" từ các phản hồi của người quản trị. Khi một cảnh báo giả được đánh dấu, mô hình sẽ tự động cập nhật trọng số để không lặp lại sai lầm đó trong tương lai.
- Triển khai thực tế: Tích hợp mô hình 1D-CNN đã huấn luyện vào các Framework xử lý luồng như Apache Spark hoặc các hệ thống IDS mã nguồn mở (như Suricata) để đánh giá hiệu năng trong môi trường thực tế.

TÀI LIỆU THAM KHẢO

- [1]. Canadian Institute for Cybersecurity (UNB). *Intrusion Detection Evaluation Dataset (CIC-IDS2017)*. <https://www.unb.ca/cic/datasets/ids-2017.html>
- [2]. K. Scarfone, P. Mell. *Guide to Intrusion Detection and Prevention Systems (IDPS)*. NIST Special Publication 800-94 Revision 1, National Institute of Standards and Technology, 2007.
- [3]. I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani. *Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization*. Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), Portugal, January 2018.
- [4]. A. H. Lashkari, G. D. Gil, M. S. I. Mamun, A. A. Ghorbani. *Characterization of Encrypted and VPN Traffic Using Time-Related*. Proceedings of the 2nd International Conference on Information Systems Security and Privacy (ICISSP), Italy, February 2016.
- [5]. N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer. *SMOTE: Synthetic Minority Over-sampling Technique*. Journal of Artificial Intelligence Research, Volume 16, pages 321–357, June 2002.
- [6]. D. P. Kingma, J. Ba. *Adam: A Method for Stochastic Optimization*. Proceedings of the 3rd International Conference on Learning Representations (ICLR), San Diego, 2015.
- [7]. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov. *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. Journal of Machine Learning Research, Volume 15, pages 1929–1958, 2014.
- [8]. Y. LeCun, Y. Bengio, G. Hinton. *Deep learning*. Nature, Volume 521, pages 436–444, May 2015.
- [9]. S. Kiranyaz, O. Avci, O. Abdeljaber, T. Ince, M. Gabbouj, D. J. Inman. *1D convolutional neural networks and applications: A survey*. Mechanical Systems and Signal Processing, Volume 151, 2021.
- [10]. V. Nair, G. E. Hinton. *Rectified Linear Units Improve Restricted Boltzmann Machines*. Proceedings of the 27th International Conference on Machine Learning (ICML), Haifa, Israel, 2010.